BMSCE Timetable Finder

Project report: a Neo4j graph database, a Node.js backend and a web front end for the college timetable

College	B.M.S. College of Engineering, Bengaluru: Department of Computer Science and Engineering
Scope	Cluster E: 5th semester (5E1-5E4) and 3rd semester (3E1-3E4), academic year 2026-27
Report date	27 October 2026
Project folder	C:\Users\dell\Desktop\project k
Repository	https://github.com/makkenavineela-dev/Timetable-Finder (local commit f969cd6, push pending, see section 14)
Technology	Neo4j 5.26 Community, Node.js 24, Express 4, plain HTML/CSS/JavaScript

Contents

1. Executive summary
2. What was done (timeline)
3. System architecture
4. The Neo4j database
5. Database connection and security
6. Data: sources, loading, assumptions
7. Cypher queries
8. The backend (Node.js / Express)
9. The frontend (web page)
10. Testing and verification
11. Running and operating the project
12. Problems found and fixed
13. Assumptions and limitations
14. Repository and delivery status

Appendix A. Optional auto-allotment script

1. Executive summary

The project answers questions about the college timetable that a printed sheet cannot answer quickly: **which rooms are free in a given period**, **where a section is at a given time**, **when a section or a faculty member is free**, and **what a room is used for during the week**. All of it is stored as a graph in Neo4j, served by a small Node.js backend and shown in a browser page.

It started from a photo of one timetable (section 5E4) and grew, step by step, into a complete system: a college-wide graph schema, loading of two real timetables from spreadsheets, a password-protected local database, a read-only API, a web page with six screens, forms for adding and changing data with automatic clash checking, and an automated test suite.

604 nodes in the graph	**2327** relationships	**19** node labels	**29** relationship types
9 sections (8 loaded + 1 added by hand)	**177** classes on the timetable	**16** rooms in 2 blocks	**43** faculty members
29 API endpoints (health + 15 read + 13 edit)	**30** Cypher queries in two files	**98** automated tests, all passing	**2** bell schedules supported

Main results

- **Graph schema** covering the whole college (College, Department, Program, Year, Semester, Section), buildings (Block, Floor, Room), courses with **L-T-P rules**, faculty, batches, classes and the weekly time grid.
- **Real data loaded** from two spreadsheets: 5th semester (80 classes) and 3rd semester (96 classes), converted by generator scripts so every assumption is written down in the data file.
- **Two bell schedules** (the 3rd semester has tea and lunch at different times). Rooms and teachers shared between the semesters are checked against each other through overlapping periods.
- **Secure database connection**: the project's own Neo4j requires a login with a generated 32-character password stored only in `backend/.env`; the API connects read-only.
- **Backend**: 15 read-only endpoints plus a guarded set of edit endpoints, input validation, rate limiting, security headers.
- **Frontend**: Floors, Free rooms, Sections, Faculty, Rooms and Add data tabs; works live against the backend and offline from a snapshot.

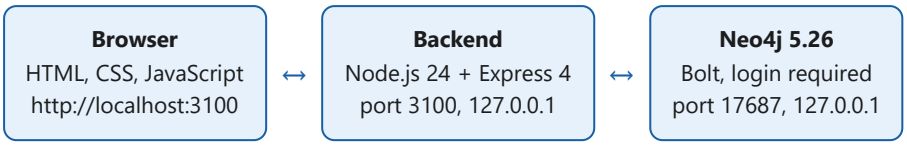
- **Verified**: every session of the 3rd semester spreadsheet (116 period cells) matches the database; zero clashes within or between schedules; 98 automated tests pass.

> **Note on the current database.** The loaded timetables contain 8 sections and 176 classes. The live database now also holds changes made by hand through the Add data tab (an extra section 5E5 with one class, some changed course teachers and classes). The statistics in this report are read from the live database, so they include those edits. `npm run load-data` restores the 8 sections to the spreadsheets.

2. What was done (timeline)

Phase	Work	Result
1. Extraction	Read the timetable image of section 5E4: rooms, subjects, codes, faculty, days and times.	Structured table of the whole week
2. Schema design	Designed a scalable graph model for 10,000+ students, many sections, branches and semesters. Added the **L-T-P rule** (credits to classes per week) and the time grid so that *free time of a section* and *free rooms on a floor* are simple queries.	`01_schema.cypher`, schema graph PDF, UML class diagram
3. Database setup	Aura and Neo4j Desktop were tried; the final design uses the project's own Neo4j Community 5.26 in `local-neo4j/`, started by a launcher.	Database on bolt port 17687
4. Real data	Spreadsheet of the 5th semester loaded with a generator (`build_cluster_E_sem5_cypher.py`); checked 80/80 sessions against the sheet.	4 sections, 80 classes
5. Backend	Node.js/Express API running the 15 read queries, with validation, error messages and tests.	Port 3100
6. Frontend	HTML/CSS/JS page: Floors, Free rooms, Sections, Faculty, Rooms; cascading pickers; merged lab cells; dark/light mode; live and offline modes.	`frontend/`
7. Hardening and clean-up	Code review and bug fixing; removal of obsolete files; password-protected database with a generated password; read-only API session; security headers; one launcher; Desktop shortcut.	57 tests
8. Add data	A tab with forms to add, change and remove classes, rooms, faculty, courses, sections and course teachers, with clash detection inside one transaction.	76 tests
9. Third semester + schedules	Second spreadsheet loaded. Because its bell schedule differs, the model gained `TimeGrid`, `FOLLOWS` and `OVERLAPS`. Drop-down lists replaced the check-box lists in the Add data forms.	8 sections, 176 classes, 98 tests
10. Delivery	Git repository prepared and committed (42 files, secrets excluded); this report.	Push pending

3. System architecture



The backend does three jobs: it serves the page files, it answers `/api/...` requests by running Cypher queries (always with parameters), and it applies changes from the Add data forms. The browser never talks to Neo4j directly. When the page is opened without the backend (double-clicking `index.html`) it shows a snapshot of the data stored in `data.js`.

3.1 Technology

Part	Technology	Why
Database	Neo4j Community 5.26.31 (Cypher 5, Bolt)	Timetable data is a network of links (section - class - room - time); free time is a missing link
Backend	Node.js 24, Express 4, neo4j-driver 5.28, dotenv, cors, express-rate-limit	Small, no build step, official Neo4j driver
Frontend	Plain HTML, CSS and JavaScript (no framework, no build)	Opens with a double-click; easy to read and change
Tests	node:test (built into Node), real database	No extra packages; tests exercise real queries
Data tools	Python 3 + openpyxl	Read spreadsheets and generate the Cypher data files
Launcher	start_timetable_finder.bat (Windows)	One double-click starts database, server and page

3.2 Folder structure

```
project k/
  start_timetable_finder.bat        one double-click start (database + server + browser)
  README.md   .gitignore            instructions; keeps passwords and the database out of Git
  Cluster_E_3rd_Sem_Timetable_Extracted.xlsx   source of the 3rd semester data
  backend/
    .env  .env.example              settings; .env holds the database password (never shared)
    package.json
    src/    server.js app.js routes.js edit.js db.js queries.js validate.js env.js
    scripts/ load-data.js export-snapshot.js
    test/   api.test.js edit.test.js load-data.test.js
  frontend/
    index.html style.css app.js edit.js api.js config.js data.js README.md
  neo4j/
    01_schema.cypher                schema, L-T-P rules, bell schedules
    02_cluster_E_sem5.cypher        5th semester data (generated)
    02_cluster_E_sem3.cypher        3rd semester data (generated)
    03_load_students.cypher         how to link 10,000+ students from a CSV
    04_queries.cypher               queries to run by hand in Neo4j Browser
    05_app_queries.cypher           the 15 read queries used by the API
    06_edit_queries.cypher          the queries behind the Add data forms
    07_auto_allot.cypher            optional automatic class placement from L-T-P
    class_diagram.mmd / .pdf        UML class diagram     01_schema_graph.pdf  schema graph
    tools/  timetable_common.py build_cluster_E_sem5_cypher.py build_cluster_E_sem3_cypher.py
  local-neo4j/                      the project's own Neo4j Community 5.26 with its data (not in Git)
```

Service	Address	Notes
Web page and API	http://localhost:3100	Backend; bound to this computer only unless HOST=0.0.0.0 is set
Neo4j Bolt	bolt://127.0.0.1:17687	Used by the backend; login required

Service	Address	Notes
Neo4j Browser	http://localhost:17474	Graph visualisation and manual Cypher; same login

4. The Neo4j database

4.1 Server

Item	Value
Product / version	Neo4j Kernel 5.26.31, Community edition
Location	local-neo4j/ inside the project (bin, conf, data, lib, plugins)
Java	Zulu JRE 21 from the Neo4j Desktop cache (Neo4j 5.26 needs Java 17 or 21; Java 24 on the PATH is too new)
Ports	Bolt 17687, HTTP 17474 (HTTPS off), both on 127.0.0.1
Memory	heap 512 MB to 1 GB, page cache 256 MB
Schema objects	20 constraints (unique keys), 35 indexes (including those the constraints create)
Size now	604 nodes, 2327 relationships

4.2 Graph model

Every real-world thing is a node; every fact about how things relate is a relationship. The same model serves one section or the whole college, because identifiers are built from the parent (a section id contains its semester id, which contains its year id, and so on) so that "5A" or "Semester 5" can repeat in every department without clashing.

> **How the timetable questions become simple:** a `Class` links to the `Section` it is for, the `Faculty` who teach it, the `Room` it uses, and to *every* `TimeSlot` it occupies. So **a section, room or teacher is free in a period when no class links to both**. Free time is not stored; it is the absence of a link, which is why it is always up to date.

4.3 Node labels in the live database

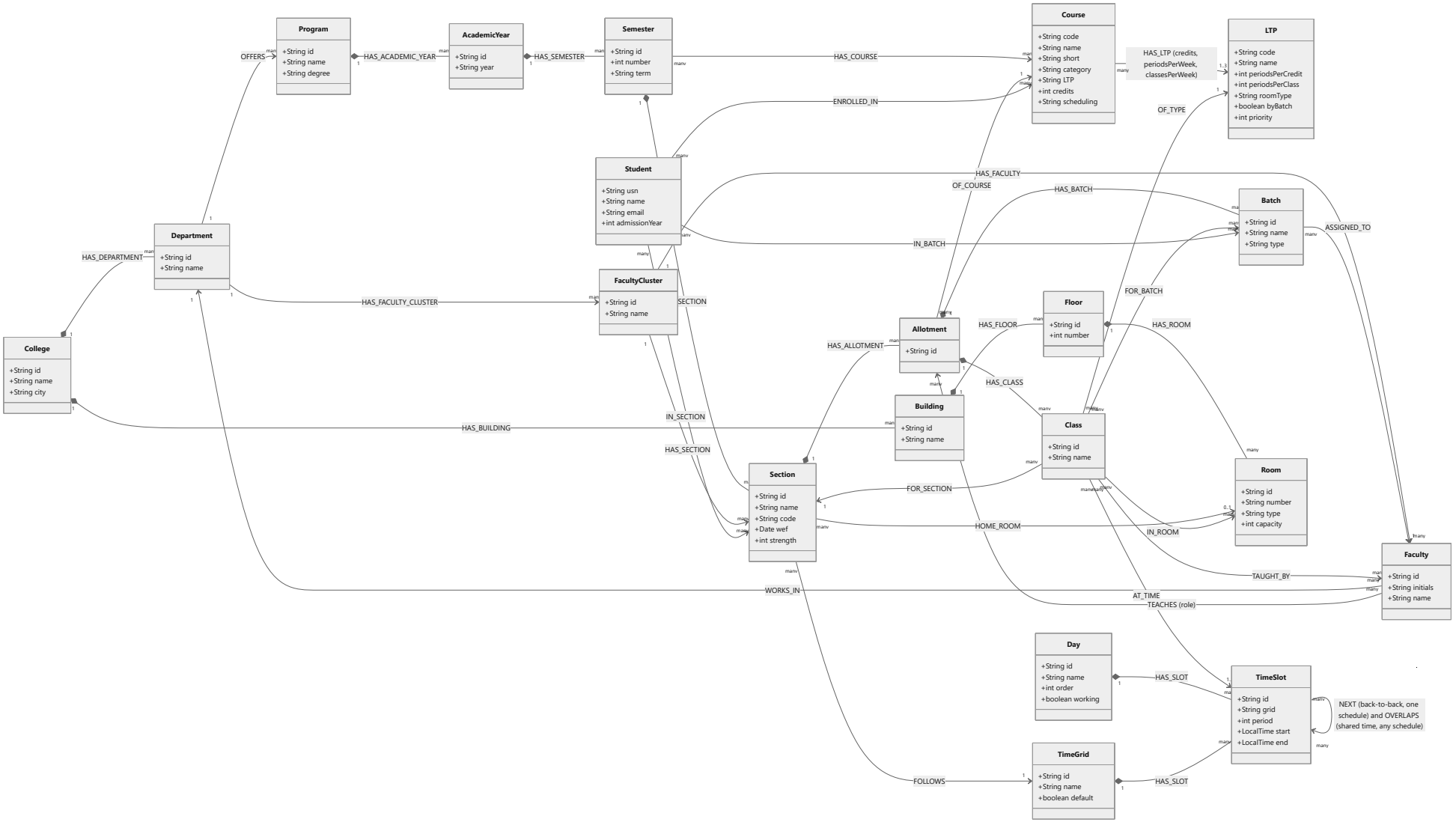
Label	Count	Meaning
College / Department / Program / AcademicYear / Semester	1 / 2 / 1 / 1 / 2	The organisation hierarchy (2 departments: CSE and Mathematics; 2 semesters: 3 and 5)
Section	9	A class group such as 5E4; follows one bell schedule
FacultyCluster	1	Cluster E of the CSE department
Faculty	43	Teachers: id = department + initials (CSE-CAV)
Course	17	Code, name, short name, L-T-P string, credits, scheduling mode
LTP	4	The four rule nodes: L Lecture, T Tutorial, P Practical, A Activity
Allotment	77	One course given to one section (carries the teacher link)
Batch	136	Lab batch B1-B4 of an allotment, each assigned to a faculty member
Class	177	One weekly class on the timetable
Building / Floor / Room	2 / 5 / 16	Blocks (PJ and 'Location not given'), floors, rooms
Day	6	MON to SAT, all working days
TimeGrid	2	A bell schedule (A and B)
TimeSlot	102	One period of one schedule on one day (schedule A: 6 days x 9 = 54, schedule B: 6 days x 8 = 48)

Student nodes are not loaded yet; `03_load_students.cypher` contains the import procedure for 10,000+ students (CSV through the Aura/Browser import tool, then linking to sections, lab batches and electives).

4.4 Relationship types in the live database

Relationship	Count	Meaning
HAS_DEPARTMENT, OFFERS, HAS_ACADEMIC_YEAR, HAS_SEMESTER, HAS_SECTION	2, 1, 1, 2, 18	Organisation hierarchy down to the section
HAS_COURSE	17	Semester to the courses of its curriculum
HAS_FACULTY_CLUSTER, HAS_FACULTY, WORKS_IN	1, 39, 43	Faculty cluster membership; department of a teacher
HAS_BUILDING, HAS_FLOOR, HAS_ROOM	2, 5, 16	Block to floor to room
HAS_LTP {credits, periodsPerWeek, classesPerWeek}	24	Course to its L, T and P parts with the numbers of classes per week
HAS_ALLOTMENT, OF_COURSE	77, 77	Section to its allotments; allotment to the course
TEACHES {role}	53	Faculty to allotment (COURSE_TEACHER)
HAS_BATCH, ASSIGNED_TO	136, 136	Allotment to lab batches; batch to its faculty
HAS_CLASS, OF_TYPE, FOR_SECTION	158, 177, 177	Class belongs to an allotment, has a type (L/T/P/A) and a section (HAS_CLASS is lower than the number of classes because activities have no course)
FOR_BATCH, TAUGHT_BY, IN_ROOM, AT_TIME	136, 238, 147, 221	Class to batches, teachers, rooms and periods (one AT_TIME per period used)
FOLLOWS	9	Section to its bell schedule
HAS_SLOT	204	Day to period and schedule to period
NEXT	66	Back-to-back periods of one schedule (never across a break)
OVERLAPS	144	Periods of any schedule sharing time on a day; every period also overlaps itself

4.5 UML class diagram of the schema



Filled diamond: the parent owns the child. Arrow: reference. Text in brackets: properties stored on the relationship.

4.6 Identifier convention

Node	Id pattern	Example
Department	code	CSE
Program	department-program	CSE-UG
AcademicYear	program-year	CSE-UG-2026-27
Semester	year-S + number	CSE-UG-2026-27-S5
Section	semester-name	CSE-UG-2026-27-S5-5E4
Allotment	section-course code	CSE-UG-2026-27-S5-5E4-23CS5PCAIN
Batch	allotment-batch	...-23CS5PCCON-B1
Class	allotment-type-first period	...-23CS5PCAIN-L-MON-P5
Faculty	department-initials	CSE-CAV
Building / Floor / Room	block / block-F + floor / block-room	PJ / PJ-F4 / PJ-406
TimeSlot	grid A: DAY-P#; other grids: letter-DAY-P#	MON-P4 (A), B-MON-P4 (B)

4.7 Unique keys and indexes (Part A of the schema)

Every node type has a uniqueness constraint on its id (which also creates an index), so loading the same file twice never duplicates anything. Additional indexes cover the fields people search by (faculty initials, section name, room type, room number, floor number, period number, schedule), and a full-text index on student and faculty names.

```
CREATE CONSTRAINT college_id         IF NOT EXISTS FOR (n:College)        REQUIRE n.id   IS UNIQUE;
CREATE CONSTRAINT department_id      IF NOT EXISTS FOR (n:Department)     REQUIRE n.id   IS UNIQUE;
CREATE CONSTRAINT program_id         IF NOT EXISTS FOR (n:Program)        REQUIRE n.id   IS UNIQUE;
CREATE CONSTRAINT academic_year_id   IF NOT EXISTS FOR (n:AcademicYear)   REQUIRE n.id   IS UNIQUE;
CREATE CONSTRAINT semester_id        IF NOT EXISTS FOR (n:Semester)       REQUIRE n.id   IS UNIQUE;
CREATE CONSTRAINT section_id         IF NOT EXISTS FOR (n:Section)        REQUIRE n.id   IS UNIQUE;
CREATE CONSTRAINT faculty_cluster_id IF NOT EXISTS FOR (n:FacultyCluster) REQUIRE n.id   IS UNIQUE;
CREATE CONSTRAINT faculty_id         IF NOT EXISTS FOR (n:Faculty)        REQUIRE n.id   IS UNIQUE;
CREATE CONSTRAINT course_code        IF NOT EXISTS FOR (n:Course)         REQUIRE n.code IS UNIQUE;
CREATE CONSTRAINT allotment_id       IF NOT EXISTS FOR (n:Allotment)      REQUIRE n.id   IS UNIQUE;
CREATE CONSTRAINT batch_id           IF NOT EXISTS FOR (n:Batch)          REQUIRE n.id   IS UNIQUE;
CREATE CONSTRAINT class_id           IF NOT EXISTS FOR (n:Class)          REQUIRE n.id   IS UNIQUE;
CREATE CONSTRAINT ltp_code           IF NOT EXISTS FOR (n:LTP)            REQUIRE n.code IS UNIQUE;
CREATE CONSTRAINT student_usn        IF NOT EXISTS FOR (n:Student)        REQUIRE n.usn  IS UNIQUE;
CREATE CONSTRAINT building_id        IF NOT EXISTS FOR (n:Building)       REQUIRE n.id   IS UNIQUE;
CREATE CONSTRAINT floor_id           IF NOT EXISTS FOR (n:Floor)          REQUIRE n.id   IS UNIQUE;
CREATE CONSTRAINT room_id            IF NOT EXISTS FOR (n:Room)           REQUIRE n.id   IS UNIQUE;
CREATE CONSTRAINT day_id             IF NOT EXISTS FOR (n:Day)            REQUIRE n.id   IS UNIQUE;
CREATE CONSTRAINT timeslot_id        IF NOT EXISTS FOR (n:TimeSlot)       REQUIRE n.id   IS UNIQUE;
CREATE CONSTRAINT timegrid_id        IF NOT EXISTS FOR (n:TimeGrid)       REQUIRE n.id   IS UNIQUE;
```

4.8 The L-T-P rule

Credits are written as L-T-P (lecture-tutorial-practical). The number of classes a course needs per week follows from its credits:

classes per week = credits x periodsPerCredit / periodsPerClass with L = 1 period per credit in one-period classes, T = 2 and 2, P = 2 and 2.

Example: AI is 3-0-1, so it needs 3 one-period lectures and 1 two-period lab per week (4 classes). DEV is 2-1-0: 2 lectures and 1 two-period tutorial. The numbers are stored on the `HAS_LTP` relationship and the rule numbers live on the four `LTP` nodes, so changing the college policy means editing one node.

```
// ---------- LTP nodes: how many classes each credit needs ----------
// Change these numbers if the college changes its credit-hour policy.
```

```
//   periodsPerCredit  periods per week for 1 credit
//   periodsPerClass   length of one class (2 = back-to-back periods)
//   roomType          room the class needs
//   byBatch           class is taken batch-wise (all batches at once, one lab)
//   priority          allotment order: hardest to fit first

UNWIND [
  {code:'L', name:'Lecture',   periodsPerCredit:1, periodsPerClass:1, roomType:'CLASSROOM', byBatch:false, priority:3},
  {code:'T', name:'Tutorial',  periodsPerCredit:2, periodsPerClass:2, roomType:'CLASSROOM', byBatch:false, priority:2},
  {code:'P', name:'Practical', periodsPerCredit:2, periodsPerClass:2, roomType:'LAB',       byBatch:true,  priority:1},
  {code:'A', name:'Activity'}
] AS rule
MERGE (ct:LTP {code: rule.code})
  SET ct += rule;
```

4.9 Bell schedules (time grids)

The 5th semester follows **schedule A** (tea 10:45-11:15, lunch 14:00-14:55, nine periods). The 3rd semester follows **schedule B** (tea 09:50-10:20, lunch 13:05-14:00, eight periods). Rooms and teachers are shared, so each period links with `OVERLAPS` to every period of the other schedule that shares time with it. **A room or teacher is busy in a period when a class of theirs is in that period or in an overlapping one.** Sections only ever use their own schedule.

Schedule	Periods	Breaks
A (default)	P1 08:00-08:55, P2 08:55-09:50, P3 09:50-10:45, P4 11:15-12:10, P5 12:10-13:05, P6 13:05-14:00, P7 14:55-15:50, P8 15:50-16:45, P9 16:45-17:40	Tea 10:45-11:15, lunch 14:00-14:55
B	P1 08:00-08:55, P2 08:55-09:50, P3 10:20-11:15, P4 11:15-12:10, P5 12:10-13:05, P6 14:00-14:55, P7 14:55-15:50, P8 15:50-16:45	Tea 09:50-10:20, lunch 13:05-14:00

```
// ---------- Bell schedules (time grids) ----------
//   A  tea 10:45-11:15, lunch 14:00-14:55, nine periods (the 5th semester sections)
//   B  tea 09:50-10:20, lunch 13:05-14:00, eight periods (the 3rd semester sections)
// default = the schedule the room screens show first.

UNWIND [
  {id:'A', name:'Schedule A (tea 10:45-11:15, lunch 14:00-14:55)', default:true},
  {id:'B', name:'Schedule B (tea 09:50-10:20, lunch 13:05-14:00)', default:false}
] AS g
MERGE (tg:TimeGrid {id: g.id})
  SET tg.name = g.name, tg.default = g.default;


// ---------- Periods: one TimeSlot per grid, day and period ----------
// Grid A keeps the ids MON-P1 ... SAT-P9, other grids add their letter: B-MON-P1 ... B-SAT-P8.

UNWIND [
  {grid:'A', periods:[[1,'08:00','08:55'], [2,'08:55','09:50'], [3,'09:50','10:45'], [4,'11:15','12:10'],
[5,'12:10','13:05'],
                    [6,'13:05','14:00'], [7,'14:55','15:50'], [8,'15:50','16:45'], [9,'16:45','17:40']]},
  {grid:'B', periods:[[1,'08:00','08:55'], [2,'08:55','09:50'], [3,'10:20','11:15'], [4,'11:15','12:10'],
[5,'12:10','13:05'],
                    [6,'14:00','14:55'], [7,'14:55','15:50'], [8,'15:50','16:45']]}
] AS gr
MATCH (tg:TimeGrid {id: gr.grid}), (day:Day)
UNWIND gr.periods AS p
MERGE (t:TimeSlot {id: CASE gr.grid WHEN 'A' THEN '' ELSE gr.grid + '-' END + day.id + '-P' + toString(p[0])})
  SET t.period = p[0], t.start = localtime(p[1]), t.end = localtime(p[2]), t.grid = gr.grid
MERGE (day)-[:HAS_SLOT]->(t)
MERGE (tg)-[:HAS_SLOT]->(t);

// NEXT joins back-to-back periods of one grid, never across the tea break or lunch,
// so a two-period lab or tutorial is always placed in continuous periods.
MATCH (tg:TimeGrid)-[:HAS_SLOT]->(a:TimeSlot)<-[:HAS_SLOT]-(day:Day)-[:HAS_SLOT]->(b:TimeSlot)<-[:HAS_SLOT]-(tg)
WHERE a.end = b.start
MERGE (a)-[:NEXT]->(b);

// OVERLAPS joins periods that share time on the same day, across grids (and each period to itself),
// so a room or faculty member busy in one schedule shows as busy in the other.
MATCH (day:Day)-[:HAS_SLOT]->(a:TimeSlot), (day)-[:HAS_SLOT]->(b:TimeSlot)
WHERE a.id < b.id AND a.start < b.end AND b.start < a.end
MERGE (a)-[:OVERLAPS]->(b);
```

```
// ... and each period to itself (the pattern above never matches the same period twice).
MATCH (t:TimeSlot)
MERGE (t)-[:OVERLAPS]->(t);
```

5. Database connection and security

5.1 How the backend connects

The backend uses the official `neo4j-driver` (Bolt). The connection code is in `backend/src/db.js`. The driver is created on first use, after `backend/.env` has been loaded by `src/env.js` (which uses a fixed path, so the server works from any folder).

Setting (backend/.env)	Value / meaning
NEO4J_URI	bolt://127.0.0.1:17687 (project database). Neo4j Desktop: neo4j://127.0.0.1:7687. Aura: neo4j+s://ID.databases.neo4j.io
NEO4J_USER	neo4j
NEO4J_PASSWORD	Generated random 32-character password (stored only in this file; never printed, never committed)
NEO4J_DATABASE	neo4j
PORT / HOST	3100 / 127.0.0.1 (set HOST=0.0.0.0 only on a trusted network)
FRONTEND_URL	Optional. Only if the page is served from another address; enables CORS for that address

5.2 Driver behaviour

- **Authentication:** `neo4j.auth.basic(user, password)`. There is no "no login" mode any more.
- **Timeouts:** connection 4 s, transaction retry 4 s, acquisition 6 s. A stopped database now gives a clear message within about 7 seconds instead of leaving the page on "Loading" for 30 seconds.
- **Integers:** `disableLosslessIntegers: true` returns whole numbers as normal JavaScript numbers. When sending data, whole numbers are converted to Neo4j integers (otherwise JavaScript sends 1 as 1.0 and a semester would be stored as a decimal; this was found and fixed by a test).
- **Read-only API:** every API query runs in a **READ session** through `session.executeRead`; Neo4j itself refuses any query that tries to change data (a test proves this).
- **Writes:** only `change(work)` opens a write transaction (`executeWrite`). A form action runs all its queries in **one transaction**: if a check fails (for example a clash), nothing is saved.
- **Parameters only:** user input is always passed as query parameters, never joined into query text.
- **Friendly errors:** `explain(err)` turns driver errors into plain-English messages (cannot reach the database, login refused, too many wrong passwords, password must be changed, database not found).

5.3 The database login

- The password was generated by a script with a random generator and set with `neo4j-admin dbms set-initial-password`. It was written directly into `backend/.env` and never displayed.
- `neo4j.conf`: `dbms.security.auth_enabled=true`, `dbms.security.auth_minimum_password_length=12`, listen address 127.0.0.1 only, usage reporting off.
- Tests confirmed that **no login, a wrong password and the default password are all refused**.
- To look at the graph: open http://localhost:17474 and sign in with `NEO4J_USER` / `NEO4J_PASSWORD` from `backend/.env`.
- To use Neo4j Desktop or Aura instead: change the three `NEO4J_*` lines, run `npm run load-data` once, start the app.

5.4 Security measures at a glance

Layer	Measure
Database	Login required; generated 32-character password; minimum length 12; listens on 127.0.0.1 only
Secrets	.env and any .env.* are in .gitignore; only .env.example (no password) is in Git; local-neo4j/ is excluded
API queries	Read-only session; parameters only; 15 fixed queries loaded from a file at start-up
Input validation	Every day, slot, schedule, id, room type, seat count and search text is checked against a strict pattern before use (see section 8.4)
Rate limit	300 requests per minute on /api
Page files	Only 6 named files are served (index.html, style.css, app.js, edit.js, api.js, data.js) plus a generated config.js; README, scripts and .env are not reachable (tested)
Headers	Content-Security-Policy (own files only, no framing), X-Content-Type-Options nosniff, Referrer-Policy no-referrer, Cache-Control no-store on the API
Editing	Accepted only from the computer running the server, only with a Host of localhost/127.0.0.1, and only if the browser's Origin is the page itself (blocks other websites and DNS-rebinding); JSON only, 20 KB limit
Front end	Page is built with DOM nodes and text, never innerHTML, so data cannot inject markup
Removal	Deletes are refused while anything still depends on the item; two clicks are needed in the page

6. Data: sources, loading, assumptions

6.1 Sources

File	Content	Used for
cluster_E_timetable_dataset.xlsx	Sheets Sessions, Subjects_Faculty, Metadata: 5th semester of cluster E, effective 28/09/2026	neo4j/02_cluster_E_sem5.cypher
Cluster_E_3rd_Sem_Timetable_Extracted.xlsx	Sheets Read Me, All Class Slots, Subjects & Credits, Faculty by Class, Lab Batch Allocations and one sheet per section (3E1-3E4)	neo4j/02_cluster_E_sem3.cypher

6.2 Pipeline

The spreadsheets are not loaded directly. A Python generator reads each workbook and writes a `.cypher` file. The shared logic is in `timetable_common.py` (time-grid slot ids, room rules, Cypher writer); each semester has its own small script that understands that sheet's layout. Every decision that goes beyond what a sheet literally says is collected and written into the header of the generated file, so it can be reviewed and changed.

Each generated file has the same blocks: **0** clean-up of earlier data of the same sections, **1** college / department / program / year / semester / cluster, **2** sections and their schedule, **3** blocks, floors and rooms, **4** courses with L-T-P, **5** faculty, **6** allotments with course teachers, **7** lab batches and their faculty, **8** classes (lectures, tutorials, labs, elective slots, activities). Everything except block 0 uses `MERGE`, so the files can be run again safely.

6.3 Loading

```
cd backend
npm run load-data     # 01_schema.cypher, 02_cluster_E_sem5.cypher, 02_cluster_E_sem3.cypher
```

`scripts/load-data.js` splits each file into statements (ignoring comments, respecting quotes), runs them with the password from `.env`, reports nodes and relationships created per file, and prints a final count. It was run twice in a row; the second run left the timetable unchanged (byte-identical snapshot). Classes added by hand to the eight loaded sections are replaced by the spreadsheet version.

6.4 Sections in the database

Section	Schedule	Classes now
3E1	B	23
3E2	B	24
3E3	B	24
3E4	B	24
5E1	A	21
5E2	A	20
5E3	A	20
5E4	A	20
5E5	A	1

6.5 Courses

Sem	Short	Name	Code	L-T-P	Scheduling
3	DBMS	Database Management Systems	26CS3IPDBM	3-0-1	REGULAR
3	DLC	Digital Logic and Computer Organization	26CS3PCDLC	3-0-0	REGULAR

Sem	Short	Name	Code	L-T-P	Scheduling
3	DMS	Discrete Mathematics and Statistics	26MA3BSDMS	3-1-0	REGULAR
3	DS	Data Structures	26CS3IPDST	3-0-1	REGULAR
3	FSD	Full Stack Development	26CS3AEFSD	0-0-1	REGULAR
3	NCMC	NCMC	26CS3NCXXX	0-0-0	NONE
3	OOP	Object Oriented Programming	26CS3PCOOP	3-0-1	REGULAR
3	Skill Lab-1	Skill Lab-1	26CS3SDXX	0-0-1	NONE
5	AI	Artificial Intelligence	23CS5PCAIN	3-0-1	REGULAR
5	BIS	Bio Inspired Systems	23CS5BSBIS	0-0-1	REGULAR
5	CC	Ability Enhancement Course	23CS5SAEXXX	0-0-1	REGULAR
5	CN	Computer Networks	23CS5PCCON	3-0-1	REGULAR
5	DEV	Data Exploration and Visualization	23CS5PCDEV	2-1-0	REGULAR
5	EVS	Environmental Studies	23CV5HSEVS	1-0-0	REGULAR
5	MIP	Mini Project	23CS5PWMIP	0-0-1	NONE
5	OOM	Object Oriented Modelling	23CS5PCOOM	2-0-1	REGULAR
5	PE	Professional Elective	23CS5SPEXXX	3-0-0	ELECTIVE_SLOT

Scheduling: REGULAR = needs a teacher and a room; ELECTIVE_SLOT = reserves the period only; NONE = not on the timetable.

6.6 Rooms

Block	Floor	Room	Type
OTHER	0	BSN-PG LAB	LAB
OTHER	0	ISE-SH	LAB
PJ	1	105	CLASSROOM
PJ	2	204	CLASSROOM
PJ	2	206	CLASSROOM
PJ	2	208	CLASSROOM
PJ	2	213	CLASSROOM
PJ	2	220	CLASSROOM
PJ	4	406	CLASSROOM
PJ	4	407	CLASSROOM
PJ	4	408	CLASSROOM
PJ	4	4F LAB 1	LAB
PJ	4	4F LAB 1-2	LAB
PJ	4	LAB 401	LAB
PJ	5	LAB 503	LAB
PJ	5	LAB 504	LAB

6.7 Assumptions made while converting the 5th semester sheet

- PE is an elective slot (reserves the period only) and Mini Project is not timetabled; both have no teacher in the file.
- The sessions called CC LAB are taken as the lab of the AE course (same assumption as in the 3rd semester file).
- Course codes follow this file: PE = 23CS5SPEXXX, AE = 23CS5SAEXXX (the earlier printed timetable showed 23CS5PEXXX / 23CS5AEXXX). The clean-up block removes the old two courses so they are not allotted twice.

- Room 'ISE-SH': block, floor and type are not given. Placed in a block called OTHER (floor 0) as a LAB, because a lab session is held there. Correct this if you know where it is.
- 'LAB 503,504' is read as 2 separate rooms (LAB 503, LAB 504); the class is linked to each of them.
- Room 'BSN-PG LAB': block, floor and type are not given. Placed in a block called OTHER (floor 0) as a LAB, because a lab session is held there. Correct this if you know where it is.
- '4F LAB 1-2' is kept as ONE room exactly as written. If it means 4F LAB 1 and 4F LAB 2 together, it overlaps the use of 4F LAB 1 by other sections.
- DEV_T sessions are the DEV tutorial (type T, two periods); the sheet lists them as 'Lecture'.
- EVS (1-0-0) has no session in the file, so it has no class yet. The allotment in Part C of 01_schema.cypher can place it.
- Effective from 28/09/2026; the earlier printed 5E4 timetable said 21/09/2026, so the file's date is used.
- Saturday is a working day (01_schema.cypher), because 5E1, 5E2 and 5E3 have Saturday classes.
- Room blocks: all numbered rooms and 'LAB nnn' are put in block PJ on the floor given by the first digit (4F LAB = floor 4). Only 406/407 were confirmed as PJ earlier; the rest are assumed.
- These sections follow time grid A: tea 10:45-11:15, lunch 14:00-14:55.

6.8 Assumptions made while converting the 3rd semester sheet

- Skill Lab-1 has a teacher but no session in the file, so it is stored as not timetabled.
- NCMC (26CS3NCXXX) has no L-T-P in the file: stored as 0-0-0 and not timetabled (a non-credit course).
- Dr. Chaitra V (teaches DMS) has no initials in the file: initials CV are made from the name, and the person is placed in the Mathematics department (DMS is a mathematics course), without a cluster.
- Dr. Shivraj D L (teaches DMS) has no initials in the file: initials SDL are made from the name, and the person is placed in the Mathematics department (DMS is a mathematics course), without a cluster.
- Dr. Rajashi Chatterjee (teaches DMS) has no initials in the file: initials RC are made from the name, and the person is placed in the Mathematics department (DMS is a mathematics course), without a cluster.
- Ms. Likhita H A (teaches DMS) has no initials in the file: initials LHA are made from the name, and the person is placed in the Mathematics department (DMS is a mathematics course), without a cluster.
- '4F LAB 1-2' is kept as ONE room exactly as written. If it means 4F LAB 1 and 4F LAB 2 together, it overlaps the use of 4F LAB 1 by other sections.
- Room 'BSN-PG LAB': block, floor and type are not given. Placed in a block called OTHER (floor 0) as a LAB, because a lab session is held there. Correct this if you know where it is.
- Seminar / Workshop / Proctor Meet on WED is written as 12:10-14:00, which runs into the break after 13:05; it is stored as 12:10-13:05.
- These sections follow time grid B: tea 09:50-10:20, lunch 13:05-14:00, eight periods (08:00 to 16:45). The file's times were written without am/pm; hours below 8 are read as afternoon.
- DMS_T sessions are the DMS tutorial (type T, two periods): DMS is 3-1-0.
- DIP-MATH (A) and DIP-MATH (B) are not in the subject list. They are stored as one-period activities without course, room or faculty (type A), like the seminar slot; (A) and (B) are kept in the name.
- The classes that run in several rooms or '4F LAB 1-2' follow the same room rules as the 5th semester file.
- Faculty names come from 'Faculty by Class'; people who only appear in lab batches have initials only. Everyone with initials is put in the CSE department and cluster E, as in the 5th semester file.
- Effective from 28/09/2026.

7. Cypher queries

All queries are kept in `.cypher` files, not in the JavaScript code. The backend reads the files at start-up and finds each query by its `// @name:` tag. This makes the queries easy to review and to run in Neo4j Browser.

File	Purpose	Queries
05_app_queries.cypher	Read-only queries behind the screens (run by the API)	15
06_edit_queries.cypher	Add, change, remove and clash checks (run by the Add data endpoints)	15
04_queries.cypher	Queries to run by hand in Neo4j Browser (reports and checks)	14
07_auto_allot.cypher	Optional automatic class placement (Appendix A)	2

How to read the queries

- `MATCH (a)-[:REL]->(b)` follows a link; `OPTIONAL MATCH` keeps the row even when the link is missing (this is how *free* is detected: the class is null).
- `NOT EXISTS { ... }` keeps only rows where a pattern is absent: *no class in this period*.
- `(:TimeSlot)-[:OVERLAPS]-(t)` means "this period or any period of the other schedule that shares time with it".
- `$name` is a parameter supplied by the backend after validation.

7.1 Read queries (05_app_queries.cypher)

R1. `meta_rooms`

Filter lists for the search screens: buildings > floors > rooms

Example parameters: `{}`

```
MATCH (b:Building)-[:HAS_FLOOR]->(f:Floor)-[:HAS_ROOM]->(r:Room)
RETURN b.id AS building, b.name AS buildingName, f.id AS floorId, f.number AS floor, r.id AS roomId,
       r.number AS room, r.type AS type, r.capacity AS capacity
ORDER BY building, floor, room;
```

R2. `meta_sections`

Example parameters: `{}`

```
MATCH (d:Department)-[:OFFERS]->(p:Program)-[:HAS_ACADEMIC_YEAR]->(ay:AcademicYear)
      -[:HAS_SEMESTER]->(sem:Semester)-[:HAS_SECTION]->(sec:Section)
OPTIONAL MATCH (fc:FacultyCluster)-[:HAS_SECTION]->(sec)
OPTIONAL MATCH (sec)-[:FOLLOWS]->(g:TimeGrid)
RETURN sec.id AS id, sec.name AS section, d.id AS department, p.name AS program,
       ay.year AS year, sem.number AS semester, sem.id AS semesterId, fc.name AS cluster, g.id AS grid
ORDER BY department, year, semester, cluster, section;
```

R3. `meta_faculty`

Example parameters: `{}`

```
MATCH (f:Faculty)-[:WORKS_IN]->(d:Department)
RETURN f.id AS id, f.initials AS initials, f.name AS name, d.id AS department
ORDER BY initials, id;
```

R4. `meta_periods`

Every period of every schedule

Example parameters: `{}`

```
MATCH (g:TimeGrid)-[:HAS_SLOT]->(t:TimeSlot)<-[:HAS_SLOT]-(d:Day)
RETURN g.id AS grid, g.name AS gridName, g.default AS defaultGrid,
       d.id AS day, d.name AS dayName, d.order AS dayOrder, d.working AS working,
       t.id AS slot, t.period AS period,
       left(toString(t.start), 5) AS start, left(toString(t.end), 5) AS end
ORDER BY grid, dayOrder, period;
```

R5. `search`

One search box for sections, rooms, faculty and blocks. Each result opens its own page.

Example parameters: `{"q": "40"}`

```
CALL () {
  MATCH (s:Section) WHERE toLower(s.name) CONTAINS toLower($q)
  RETURN 'SECTION' AS kind, s.id AS id, s.name AS label
  UNION
  MATCH (r:Room) WHERE toLower(r.number) CONTAINS toLower($q) OR toLower(r.id) CONTAINS toLower($q)
  RETURN 'ROOM' AS kind, r.id AS id, r.number AS label
  UNION
  MATCH (f:Faculty) WHERE toLower(f.name) CONTAINS toLower($q) OR toLower(f.initials) CONTAINS toLower($q)
  RETURN 'FACULTY' AS kind, f.id AS id, f.name AS label
  UNION
  MATCH (b:Building) WHERE toLower(b.name) CONTAINS toLower($q) OR toLower(b.id) CONTAINS toLower($q)
  RETURN 'BUILDING' AS kind, b.id AS id, b.name AS label
}
RETURN kind, id, label
ORDER BY kind, label, id
LIMIT 25;
```

R6. `floor_occupancy`

Every room on a floor in one period: FREE, or OCCUPIED with section, class and faculty. A room used by two overlapping classes shows the first one (by class id).

Example parameters: `{"floor": "PJ-F4", "slot": "MON-P4"}`

```
MATCH (:Floor {id: $floor})-[:HAS_ROOM]->(r:Room), (t:TimeSlot {id: $slot})
OPTIONAL MATCH (r)<-[:IN_ROOM]-(cl:Class)-[:AT_TIME]->(:TimeSlot)-[:OVERLAPS]-(t)
OPTIONAL MATCH (cl)-[:FOR_SECTION]->(sec:Section)
OPTIONAL MATCH (cl)-[:TAUGHT_BY]->(f:Faculty)
WITH r, cl, sec, collect(DISTINCT f.initials) AS faculty
ORDER BY cl.id
WITH r, collect({cl: cl, sec: sec, faculty: faculty})[0] AS u
RETURN r.number AS room, r.id AS roomId, r.type AS type,
       CASE WHEN u.cl IS NULL THEN 'FREE' ELSE 'OCCUPIED' END AS status,
       u.sec.name AS section, u.sec.id AS sectionId, u.cl.name AS class, u.faculty AS faculty
ORDER BY room, roomId;
```

R7. `floor_day_grid`

Rooms x periods of one schedule (grid null = the default one) for one day: the whole floor at a glance

Example parameters: `{"floor": "PJ-F4", "day": "MON", "grid": null}`

```
MATCH (:Floor {id: $floor})-[:HAS_ROOM]->(r:Room),
      (g:TimeGrid)-[:HAS_SLOT]->(t:TimeSlot)<-[:HAS_SLOT]-(:Day {id: $day})
WHERE g.id = $grid OR ($grid IS NULL AND g.default)
OPTIONAL MATCH (r)<-[:IN_ROOM]-(cl:Class)-[:AT_TIME]->(:TimeSlot)-[:OVERLAPS]-(t)
OPTIONAL MATCH (cl)-[:FOR_SECTION]->(sec:Section)
WITH r, t, cl, sec ORDER BY cl.id
WITH r, t, collect({cl: cl, sec: sec})[0] AS u
RETURN r.number AS room, r.id AS roomId, r.type AS type, t.period AS period,
       CASE WHEN u.cl IS NULL THEN 'FREE' ELSE 'OCCUPIED' END AS status,
       u.sec.name AS section, u.sec.id AS sectionId, u.cl.name AS class
ORDER BY room, roomId, period;
```

R8. `free_rooms`

Find free rooms. Any filter can be null to skip it.

Example parameters: `{"building": "PJ", "floor": null, "type": "CLASSROOM", "slot": "MON-P2", "minCapacity": null}`

```
MATCH (b:Building)-[:HAS_FLOOR]->(fl:Floor)-[:HAS_ROOM]->(r:Room), (t:TimeSlot {id: $slot})
WHERE ($building IS NULL OR b.id = $building)
  AND ($floor IS NULL OR fl.id = $floor)
  AND ($type IS NULL OR r.type = $type)
  AND ($minCapacity IS NULL OR r.capacity >= $minCapacity)
  AND NOT EXISTS { MATCH (r)<-[:IN_ROOM]-(:Class)-[:AT_TIME]->(:TimeSlot)-[:OVERLAPS]-(t) }
RETURN b.id AS building, fl.number AS floor, r.number AS room, r.id AS roomId, r.type AS type, r.capacity AS capacity
ORDER BY building, floor, room, roomId;
```

R9. `section_week`

Week grid of a section, in the section's own schedule: each period holds a class, or nothing (free)

Example parameters: `{"section": "CSE-UG-2026-27-S5-5E4"}`

```
MATCH (sec:Section {id: $section})-[:FOLLOWS]->(:TimeGrid)-[:HAS_SLOT]->(t:TimeSlot)<-[:HAS_SLOT]-(d:Day {working:
true})
OPTIONAL MATCH (sec)<-[:FOR_SECTION]-(cl:Class)-[:AT_TIME]->(t)
OPTIONAL MATCH (cl)-[:IN_ROOM]->(r:Room)
OPTIONAL MATCH (cl)-[:OF_TYPE]->(lt:LTP)
OPTIONAL MATCH (cl)-[:TAUGHT_BY]->(f:Faculty)
WITH d, t, cl, lt, collect(DISTINCT f.initials) AS faculty, collect(DISTINCT r {.id, .number}) AS rooms
RETURN d.id AS day, d.order AS dayOrder, t.period AS period,
       left(toString(t.start), 5) AS start, left(toString(t.end), 5) AS end,
       cl.id AS classId, cl.name AS class, lt.code AS type, rooms, faculty
ORDER BY dayOrder, period;
```

R10. `section_free_time`

Free periods of a section, one day or the whole week (day = null)

Example parameters: `{"section": "CSE-UG-2026-27-S5-5E4", "day": null}`

```
MATCH (sec:Section {id: $section})-[:FOLLOWS]->(:TimeGrid)-[:HAS_SLOT]->(t:TimeSlot)<-[:HAS_SLOT]-(d:Day {working:
true})
WHERE ($day IS NULL OR d.id = $day)
  AND NOT EXISTS { MATCH (sec)<-[:FOR_SECTION]-(:Class)-[:AT_TIME]->(t) }
WITH d, t ORDER BY t.period
WITH d, collect({slot: t.id, period: t.period,
                 time: left(toString(t.start), 5) + '-' + left(toString(t.end), 5)}) AS free
RETURN d.id AS day, d.name AS dayName, free
ORDER BY d.order;
```

R11. `section_free_with_rooms`

Where an extra class could go: periods when the section is free and which classrooms are free then

Example parameters: `{"section": "CSE-UG-2026-27-S5-5E4", "day": "MON", "floor": "PJ-F4"}`

```
MATCH (sec:Section {id: $section})-[:FOLLOWS]->(:TimeGrid)-[:HAS_SLOT]->(t:TimeSlot)<-[:HAS_SLOT]-(:Day {id: $day})
WHERE NOT EXISTS { MATCH (sec)<-[:FOR_SECTION]-(:Class)-[:AT_TIME]->(t) }
OPTIONAL MATCH (:Floor {id: $floor})-[:HAS_ROOM]->(r:Room {type: 'CLASSROOM'})
WHERE NOT EXISTS { MATCH (r)<-[:IN_ROOM]-(:Class)-[:AT_TIME]->(:TimeSlot)-[:OVERLAPS]-(t) }
WITH t, r ORDER BY r.number
RETURN t.id AS slot, t.period AS period,
       left(toString(t.start), 5) + '-' + left(toString(t.end), 5) AS time,
       collect(r.number) AS freeRooms
ORDER BY period;
```

R12. `faculty_week`

Everything a faculty member teaches in the week (labs show their own batches). grid says which schedule the class's periods belong to.

Example parameters: `{"faculty": "CSE-CAV"}`

```
MATCH (f:Faculty {id: $faculty})<-[:TAUGHT_BY]-(cl:Class)-[:AT_TIME]->(t:TimeSlot)<-[:HAS_SLOT]-(d:Day)
OPTIONAL MATCH (cl)-[:FOR_SECTION]->(sec:Section)
OPTIONAL MATCH (cl)-[:FOR_BATCH]->(b:Batch)-[:ASSIGNED_TO]->(f)
WITH d, cl, sec, t, b ORDER BY b.name
WITH d, cl, sec, t.grid AS grid, min(t.period) AS firstPeriod, max(t.period) AS lastPeriod,
     min(t.start) AS start, max(t.end) AS end, collect(DISTINCT b.name) AS batches
OPTIONAL MATCH (cl)-[:IN_ROOM]->(r:Room)
WITH d, cl, sec, grid, firstPeriod, lastPeriod, start, end, batches, r ORDER BY r.number
WITH d, cl, sec, grid, firstPeriod, lastPeriod, start, end, batches, collect(r {.id, .number}) AS rooms
RETURN d.id AS day, d.order AS dayOrder, grid, firstPeriod, lastPeriod,
       left(toString(start), 5) + '-' + left(toString(end), 5) AS time,
       left(toString(start), 5) AS start, left(toString(end), 5) AS end,
       cl.name AS class, sec.name AS section, sec.id AS sectionId, rooms, batches
ORDER BY dayOrder, start, class;
```

R13. `faculty_free_time`

Periods of one schedule (grid null = the default one) in which the faculty member has no class, also not in an overlapping period of the other schedule

Example parameters: `{"faculty": "CSE-CAV", "day": "MON", "grid": null}`

```
MATCH (f:Faculty {id: $faculty}), (g:TimeGrid)-[:HAS_SLOT]->(t:TimeSlot)<-[:HAS_SLOT]-(d:Day {working: true})
WHERE (g.id = $grid OR ($grid IS NULL AND g.default))
  AND ($day IS NULL OR d.id = $day)
  AND NOT EXISTS { MATCH (f)<-[:TAUGHT_BY]-(:Class)-[:AT_TIME]->(:TimeSlot)-[:OVERLAPS]-(t) }
WITH d, t ORDER BY t.period
WITH d, collect({slot: t.id, period: t.period,
                 time: left(toString(t.start), 5) + '-' + left(toString(t.end), 5)}) AS free
RETURN d.id AS day, d.name AS dayName, free
ORDER BY d.order;
```

R14. `room_week`

A room across the week: which section uses it and when (grid = the schedule of that class)

Example parameters: `{"room": "PJ-406"}`

```
MATCH (r:Room {id: $room})<-[:IN_ROOM]-(cl:Class)-[:AT_TIME]->(t:TimeSlot)<-[:HAS_SLOT]-(d:Day)
OPTIONAL MATCH (cl)-[:FOR_SECTION]->(sec:Section)
OPTIONAL MATCH (cl)-[:TAUGHT_BY]->(f:Faculty)
WITH d, cl, sec, t, collect(DISTINCT f.initials) AS faculty
WITH d, cl, sec, faculty, t.grid AS grid, min(t.period) AS firstPeriod, max(t.period) AS lastPeriod,
     min(t.start) AS start, max(t.end) AS end
RETURN d.id AS day, d.order AS dayOrder, grid, firstPeriod, lastPeriod,
       left(toString(start), 5) + '-' + left(toString(end), 5) AS time,
       left(toString(start), 5) AS start, left(toString(end), 5) AS end,
       cl.name AS class, sec.name AS section, sec.id AS sectionId, faculty
ORDER BY dayOrder, start, class;
```

R15. `course_summary`

Courses of a section with L-T-P, teacher and how many classes are allotted

Example parameters: `{"section": "CSE-UG-2026-27-S5-5E4"}`

```
MATCH (sec:Section {id: $section})-[:HAS_ALLOTMENT]->(a:Allotment)-[:OF_COURSE]->(c:Course)
OPTIONAL MATCH (f:Faculty)-[:TEACHES {role: 'COURSE_TEACHER'}]->(a)
RETURN c.code AS code, c.short AS course, c.name AS name, c.LTP AS LTP, c.credits AS credits,
       c.scheduling AS scheduling, collect(DISTINCT f.initials) AS teacher,
       COUNT { (a)-[:HAS_CLASS]->() } AS classesAllotted,
       reduce(n = 0, x IN [(c)-[q:HAS_LTP]->() | q.classesPerWeek] | n + x) AS classesNeeded
ORDER BY course, code;
```

7.2 Edit queries (06_edit_queries.cypher)

These run only inside the Add data endpoints, inside one write transaction. "Save" queries are **upserts**: saving something that exists updates it (found by block + room number, department + initials, course code, or section id); optional values that are null keep what is stored.

E1. `courses`

Every course with its semesters and L-T-P parts (for the course, class and remove forms)

Example parameters: `{}`

```
MATCH (c:Course)
OPTIONAL MATCH (sem:Semester)-[:HAS_COURSE]->(c)
WITH c, collect(sem.id) AS semesters
RETURN c.code AS code, c.short AS short, c.name AS name, c.category AS category, c.LTP AS LTP,
       c.scheduling AS scheduling, semesters, [(c)-[:HAS_LTP]->(l:LTP) | l.code] AS parts
ORDER BY short, code;
```

E2. `save_room`

A room is found by block + room number; a new block or floor is created when needed. a room is on one floor: moving it removes the old link

Example parameters: `{"block": "PJ", "blockName": null, "floor": 4, "number": "409", "type": "CLASSROOM", "capacity": 60}`

```
MERGE (col:College {id: 'BMSCE'})
  ON CREATE SET col.name = 'B.M.S. College of Engineering', col.city = 'Bengaluru'
MERGE (b:Building {id: $block})
  ON CREATE SET b.name = coalesce($blockName, $block)
SET b.name = coalesce($blockName, b.name)
MERGE (col)-[:HAS_BUILDING]->(b)
MERGE (fl:Floor {id: $block + '-F' + toString($floor)})
  ON CREATE SET fl.number = $floor
MERGE (b)-[:HAS_FLOOR]->(fl)
WITH b, fl
OPTIONAL MATCH (b)-[:HAS_FLOOR]->(:Floor)-[:HAS_ROOM]->(same:Room {number: $number})
WITH fl, coalesce(head(collect(same.id)), $block + '-' + replace($number, ' ', '')) AS roomId
WITH fl, roomId, NOT EXISTS { MATCH (:Room {id: roomId}) } AS created
MERGE (r:Room {id: roomId})
SET r.number = $number, r.type = $type, r.capacity = coalesce($capacity, r.capacity)
WITH fl, r, created
OPTIONAL MATCH (other:Floor)-[h:HAS_ROOM]->(r) WHERE other <> fl
DELETE h
WITH DISTINCT fl, r, created
MERGE (fl)-[:HAS_ROOM]->(r)
RETURN r.id AS id, r.number AS number, fl.id AS floor, created;
```

E3. `save_faculty`

a cluster given moves the faculty member to it; no cluster keeps the current one

Example parameters: `{"department": "CSE", "initials": "ABC", "name": "Prof. A B C", "cluster": "E"}`

```
MATCH (d:Department {id: $department})
WITH d, d.id + '-' + $initials AS facultyId
WITH d, facultyId, NOT EXISTS { MATCH (:Faculty {id: facultyId}) } AS created
MERGE (f:Faculty {id: facultyId})
SET f.initials = $initials, f.name = $name
MERGE (f)-[:WORKS_IN]->(d)
WITH d, f, created
OPTIONAL MATCH (oc:FacultyCluster)-[h:HAS_FACULTY]->(f)
WHERE $cluster IS NOT NULL AND oc.id <> d.id + '-' + $cluster
DELETE h
WITH DISTINCT d, f, created
FOREACH (_ IN CASE WHEN $cluster IS NULL THEN [] ELSE [1] END |
  MERGE (fc:FacultyCluster {id: d.id + '-' + $cluster})
    ON CREATE SET fc.name = $cluster
  MERGE (d)-[:HAS_FACULTY_CLUSTER]->(fc)
  MERGE (fc)-[:HAS_FACULTY]->(f))
RETURN f.id AS id, f.name AS name, created;
```

E4. `save_course`

The L-T-P becomes HAS_LTP links (same rule as the 02_* files), and every section of the semester gets an allotment of the course.

Example parameters: `{"semester": "CSE-UG-2026-27-S5", "code": "23CS5PEABC", "name": "Cloud Computing", "short": "CLC", "category": "PE", "ltp": "3-0-0", "scheduling": "REGULAR"}`

```
MATCH (sem:Semester {id: $semester})
WITH sem, NOT EXISTS { MATCH (:Course {code: $code}) } AS created, [x IN split($ltp, '-') | toInteger(x)] AS ltp
MERGE (c:Course {code: $code})
SET c.name = $name, c.short = $short, c.category = coalesce($category, c.category), c.LTP = $ltp,
    c.credits = ltp[0] + ltp[1] + ltp[2], c.scheduling = $scheduling
MERGE (sem)-[:HAS_COURSE]->(c)
WITH sem, c, ltp, created
OPTIONAL MATCH (c)-[old:HAS_LTP]->()
DELETE old
WITH DISTINCT sem, c, ltp, created
CALL (c, ltp) {
  UNWIND [['L', ltp[0]], ['T', ltp[1]], ['P', ltp[2]]] AS part
  WITH c, part WHERE part[1] > 0
  MATCH (l:LTP {code: part[0]})
  MERGE (c)-[r:HAS_LTP]->(l)
  SET r.credits = part[1], r.periodsPerWeek = part[1] * l.periodsPerCredit,
      r.classesPerWeek = part[1] * l.periodsPerCredit / l.periodsPerClass
}
CALL (sem, c) {
  MATCH (sem)-[:HAS_SECTION]->(sec:Section)
  MERGE (a:Allotment {id: sec.id + '-' + c.code})
  MERGE (sec)-[:HAS_ALLOTMENT]->(a)
  MERGE (a)-[:OF_COURSE]->(c)
}
RETURN c.code AS id, c.short AS short, c.name AS name, created;
```

E5. `save_section`

Creates whatever is missing of College > Department > Program > Year > Semester > Section, puts the section on a bell schedule (grid null = the default one, or the one it has already), and gives the section an allotment of every course of its semester.

Example parameters: `{"department": "CSE", "departmentName": null, "program": "UG", "year": "2026-27", "semester": 5, "cluster": "E", "name": "5E5", "code": null, "grid": null}`

```
MERGE (col:College {id: 'BMSCE'})
  ON CREATE SET col.name = 'B.M.S. College of Engineering', col.city = 'Bengaluru'
MERGE (d:Department {id: $department})
  ON CREATE SET d.name = coalesce($departmentName, $department)
SET d.name = coalesce($departmentName, d.name)
MERGE (col)-[:HAS_DEPARTMENT]->(d)
MERGE (p:Program {id: $department + '-' + $program})
  ON CREATE SET p.name = $program
MERGE (d)-[:OFFERS]->(p)
MERGE (ay:AcademicYear {id: p.id + '-' + $year})
  ON CREATE SET ay.year = $year
MERGE (p)-[:HAS_ACADEMIC_YEAR]->(ay)
MERGE (sem:Semester {id: ay.id + '-S' + toString($semester)})
  ON CREATE SET sem.number = $semester, sem.term = CASE WHEN $semester % 2 = 1 THEN 'ODD' ELSE 'EVEN' END
MERGE (ay)-[:HAS_SEMESTER]->(sem)
WITH d, sem, sem.id + '-' + $name AS sectionId
WITH d, sem, sectionId, NOT EXISTS { MATCH (:Section {id: sectionId}) } AS created
MERGE (sec:Section {id: sectionId})
SET sec.name = $name, sec.code = coalesce($code, sec.code)
MERGE (sem)-[:HAS_SECTION]->(sec)
WITH d, sem, sec, created
OPTIONAL MATCH (oc:FacultyCluster)-[h:HAS_SECTION]->(sec)
WHERE $cluster IS NOT NULL AND oc.id <> d.id + '-' + $cluster
DELETE h
WITH DISTINCT d, sem, sec, created
FOREACH (_ IN CASE WHEN $cluster IS NULL THEN [] ELSE [1] END |
  MERGE (fc:FacultyCluster {id: d.id + '-' + $cluster})
    ON CREATE SET fc.name = $cluster
  MERGE (d)-[:HAS_FACULTY_CLUSTER]->(fc)
  MERGE (fc)-[:HAS_SECTION]->(sec))
WITH sem, sec, created
CALL (sec) {
  MATCH (g:TimeGrid) WHERE g.id = $grid OR ($grid IS NULL AND NOT EXISTS { (sec)-[:FOLLOWS]->() } AND g.default)
  OPTIONAL MATCH (sec)-[old:FOLLOWS]->(other:TimeGrid) WHERE other <> g
  DELETE old
  MERGE (sec)-[:FOLLOWS]->(g)
}
CALL (sem, sec) {
  MATCH (sem)-[:HAS_COURSE]->(c:Course)
  MERGE (a:Allotment {id: sec.id + '-' + c.code})
  MERGE (sec)-[:HAS_ALLOTMENT]->(a)
  MERGE (a)-[:OF_COURSE]->(c)
}
RETURN sec.id AS id, sec.name AS name, created, [(sec)-[:FOLLOWS]->(g:TimeGrid) | g.id][0] AS grid;
```

E6. `set_teacher`

The course teacher of one section's course (faculty null = no teacher). Classes keep their faculty.

Example parameters: `{"section": "CSE-UG-2026-27-S5-5E4", "course": "23CS5PCAIN", "faculty": "CSE-CAV"}`

```
MATCH (sec:Section {id: $section})-[:HAS_ALLOTMENT]->(a:Allotment)-[:OF_COURSE]->(c:Course {code: $course})
OPTIONAL MATCH (:Faculty)-[old:TEACHES {role: 'COURSE_TEACHER'}]->(a)
DELETE old
WITH DISTINCT sec, a, c
OPTIONAL MATCH (f:Faculty {id: $faculty})
FOREACH (_ IN CASE WHEN f IS NULL THEN [] ELSE [1] END | MERGE (f)-[:TEACHES {role: 'COURSE_TEACHER'}]->(a))
RETURN sec.name AS section, c.short AS course, f.initials AS teacher;
```

E7. `class_lookup`

Everything the backend checks before adding a class (no rows = no such section)

Example parameters: `{"section": "CSE-UG-2026-27-S5-5E4", "kind": "L", "course": "23CS5PCAIN", "day": "MON", "period": 1, "rooms": ["PJ-406"], "faculty": ["CSE-CAV"]}`

```
MATCH (sec:Section {id: $section}), (lt:LTP {code: $kind})
OPTIONAL MATCH (sec)-[:FOLLOWS]->(:TimeGrid)-[:HAS_SLOT]->(t1:TimeSlot {period: $period})<-[:HAS_SLOT]-(d:Day {id:
$day})
OPTIONAL MATCH (t1)-[:NEXT]->(t2:TimeSlot)
OPTIONAL MATCH (sec)-[:HAS_ALLOTMENT]->(a:Allotment)-[:OF_COURSE]->(c:Course {code: $course})
RETURN sec.name AS section, d.working AS working, t1.id AS first, t2.id AS second,
       coalesce(lt.periodsPerClass, 1) AS length, lt.name AS kindName,
       a.id AS allotment, c.short AS short, c.LTP AS LTP,
       c IS NOT NULL AND EXISTS { MATCH (c)-[:HAS_LTP]->(lt) } AS inLtp,
       COLLECT { MATCH (r:Room) WHERE r.id IN $rooms RETURN r.id } AS rooms,
       COLLECT { MATCH (f:Faculty) WHERE f.id IN $faculty RETURN f.id } AS faculty;
```

E8. `class_clashes`

Classes already in those periods that use the same section, a chosen room or a chosen faculty member. Rooms and faculty also clash with classes in overlapping periods of the other schedule.

Example parameters: `{"section": "CSE-UG-2026-27-S5-5E4", "slots": ["MON-P1"], "rooms": ["PJ-406"], "faculty": ["CSE-CAV"]}`

```
UNWIND $slots AS s
MATCH (t:TimeSlot {id: s})
CALL (t) {
  MATCH (t)<-[:AT_TIME]-(cl:Class) RETURN cl, true AS ownGrid
  UNION
  MATCH (t)-[:OVERLAPS]-(:TimeSlot)<-[:AT_TIME]-(cl:Class) RETURN cl, false AS ownGrid
}
WITH DISTINCT t, cl
OPTIONAL MATCH (cl)-[:FOR_SECTION]->(sec:Section)
WITH t, cl, sec,
     COLLECT { MATCH (cl)-[:IN_ROOM]->(r:Room) WHERE r.id IN $rooms RETURN r.number } AS rooms,
     COLLECT { MATCH (cl)-[:TAUGHT_BY]->(f:Faculty) WHERE f.id IN $faculty RETURN f.initials } AS faculty
WHERE (sec.id = $section AND EXISTS { (cl)-[:AT_TIME]->(t) }) OR size(rooms) > 0 OR size(faculty) > 0
RETURN t.id AS slot, cl.name AS class, sec.name AS section, sec.id = $section AS sameSection, rooms, faculty
ORDER BY slot, class;
```

E9. `class_create`

---------- Removing. Each returns label and uses; something still in use (uses > 0) is not deleted and the backend says what depends on it. ----------

Example parameters: `{"id": "...-23CS5PCAIN-L-MON-P1", "name": "AI", "section": "CSE-UG-2026-27-S5-5E4", "kind": "L", "allotment": "CSE-UG-2026-27-S5-5E4-23CS5PCAIN", "slots": ["MON-P1"], "rooms": ["PJ-406"], "faculty": ["CSE-CAV"]}`

```
MATCH (sec:Section {id: $section}), (lt:LTP {code: $kind})
MERGE (cl:Class {id: $id})
SET cl.name = $name
MERGE (cl)-[:OF_TYPE]->(lt)
MERGE (cl)-[:FOR_SECTION]->(sec)
WITH cl
OPTIONAL MATCH (a:Allotment {id: $allotment})
FOREACH (_ IN CASE WHEN a IS NULL THEN [] ELSE [1] END | MERGE (a)-[:HAS_CLASS]->(cl))
WITH cl
CALL (cl) { UNWIND $slots AS s MATCH (t:TimeSlot {id: s}) MERGE (cl)-[:AT_TIME]->(t) }
CALL (cl) { UNWIND $rooms AS r MATCH (rm:Room {id: r}) MERGE (cl)-[:IN_ROOM]->(rm) }
CALL (cl) { UNWIND $faculty AS f MATCH (fa:Faculty {id: f}) MERGE (cl)-[:TAUGHT_BY]->(fa) }
RETURN cl.id AS id, cl.name AS name;
```

E10. `remove_class`

Example parameters: `{"id": "CSE-UG-2026-27-S5-5E4-23CS5PCAIN-L-MON-P5"}`

```
MATCH (cl:Class {id: $id})
OPTIONAL MATCH (cl)-[:FOR_SECTION]->(sec:Section)
WITH cl, cl.name + coalesce(' of ' + sec.name, '') AS label
DETACH DELETE cl
RETURN label, 0 AS uses;
```

E11. `remove_room`

Example parameters: `{"id": "PJ-409"}`

```
MATCH (r:Room {id: $id})
WITH r, 'Room ' + r.number AS label, COUNT { (r)<-[:IN_ROOM]-(:Class) } AS uses
FOREACH (_ IN CASE WHEN uses = 0 THEN [1] ELSE [] END | DETACH DELETE r)
RETURN label, uses;
```

E12. `remove_faculty`

Example parameters: `{"id": "CSE-ABC"}`

```
MATCH (f:Faculty {id: $id})
WITH f, f.name AS label,
     COUNT { (f)<-[:TAUGHT_BY]-(:Class) } + COUNT { (f)<-[:ASSIGNED_TO]-(:Batch) } + COUNT { (f)-[:TEACHES]->() } AS
uses
FOREACH (_ IN CASE WHEN uses = 0 THEN [1] ELSE [] END | DETACH DELETE f)
RETURN label, uses;
```

E13. `remove_course`

Also removes the course's allotments (and their batches) in every section.

Example parameters: `{"id": "23CS5PEABC"}`

```
MATCH (c:Course {code: $id})
WITH c, c.short + ' (' + c.name + ')' AS label, COUNT { (c)<-[:OF_COURSE]-(:Allotment)-[:HAS_CLASS]->(:Class) } AS uses
CALL (c, uses) {
  MATCH (a:Allotment)-[:OF_COURSE]->(c) WHERE uses = 0
  OPTIONAL MATCH (a)-[:HAS_BATCH]->(b:Batch)
  DETACH DELETE b, a
}
FOREACH (_ IN CASE WHEN uses = 0 THEN [1] ELSE [] END | DETACH DELETE c)
RETURN label, uses;
```

E14. `remove_section`

Also removes the section's allotments and batches.

Example parameters: `{"id": "CSE-UG-2026-27-S5-5E5"}`

```
MATCH (sec:Section {id: $id})
WITH sec, 'Section ' + sec.name AS label,
     COUNT { (sec)<-[:FOR_SECTION]-(:Class) } + COUNT { (sec)<-[:IN_SECTION]-(:Student) } AS uses
CALL (sec, uses) {
  MATCH (sec)-[:HAS_ALLOTMENT]->(a:Allotment) WHERE uses = 0
  OPTIONAL MATCH (a)-[:HAS_BATCH]->(b:Batch)
  DETACH DELETE b, a
}
FOREACH (_ IN CASE WHEN uses = 0 THEN [1] ELSE [] END | DETACH DELETE sec)
RETURN label, uses;
```

E15. `remove_empty_places`

After a room is removed: floors without rooms and blocks without floors go too

Example parameters: `{}`

```
CALL () { MATCH (fl:Floor) WHERE NOT EXISTS { (fl)-[:HAS_ROOM]->() } DETACH DELETE fl }
CALL () { MATCH (b:Building) WHERE NOT EXISTS { (b)-[:HAS_FLOOR]->() } DETACH DELETE b }
RETURN true AS done;
```

7.3 Queries for Neo4j Browser (04_queries.cypher)

Ids used in the file: section `CSE-UG-2026-27-S5-5E4`, day MON, period MON-P4, floor PJ-F4, faculty CSE-CAV. Change them as needed. All of them were run against the live database and execute without error.

#	Query
0	Draw the schema graph (labels and relationship types)
1	Required vs allotted classes per course, from L-T-P
2	Day view of a section: every period with its class, or FREE
3	Only the free periods of a section on one day
4	Free periods of a section for every working day of the week
5	Free rooms on a floor in one period (the type column shows CLASSROOM or LAB)
6	Free periods of every room on a floor for one day (in schedule A; use TimeGrid {id:'B'} for the other)
7	Classrooms on a floor that are free while the section is free
8	Free periods of a faculty member on one day (schedule A; also not busy in an overlapping period of schedule B)
9	Timetable of a section (rebuilds the printed sheet)
10	Timetable of a faculty member
11	Timetable of a student (only their own lab batch)
12	Clashes: a section, faculty member or room with two classes at the same time.
13	Sections and students per department for a year

Example: clash report (query 12), schedule-aware

Reports a section, faculty member or room with two classes at the same time, also across the two schedules. **No rows means no clashes.** The current data returns no rows.

```
// 12. Clashes: a section, faculty member or room with two classes at the same time.
//     Faculty and rooms are also checked against the other schedule's overlapping periods. No rows = no clashes.
MATCH (sec:Section)<-[:FOR_SECTION]-(cl:Class)-[:AT_TIME]->(t:TimeSlot)
WITH 'SECTION' AS kind, sec.id AS who, t, collect(cl.id) AS classes
WHERE size(classes) > 1
RETURN kind, who, t.id AS slot, classes
UNION
MATCH (x)<-[:TAUGHT_BY|IN_ROOM]-(c1:Class)-[:AT_TIME]->(t1:TimeSlot)-[:OVERLAPS]-(t2:TimeSlot)<-[:AT_TIME]-(c2:Class)-
[:TAUGHT_BY|IN_ROOM]->(x)
WHERE c1.id < c2.id
RETURN CASE WHEN x:Faculty THEN 'FACULTY' ELSE 'ROOM' END AS kind, x.id AS who, t1.id + ' / ' + t2.id AS slot, [c1.id,
c2.id] AS classes;
```

8. The backend (Node.js / Express)

8.1 Files

File	Role
src/server.js	Starts the server on 127.0.0.1:3100, pings the database and reports; <code>--open</code> opens the browser; detects an already-running copy; closes cleanly on Ctrl+C
src/app.js	Builds the Express app: security headers, CORS (only if FRONTEND_URL is set), rate limit, /api/health, routers, generated /config.js, whitelisted page files, error handler
src/routes.js	The 15 read endpoints; every handler goes through a wrapper that forwards async errors (Express 4 does not)
src/edit.js	The Add data endpoints, their access guard and the clash logic
src/db.js	Driver, read(), change(), writeSession(), ping(), explain()
src/queries.js	Loads 05_ and 06_ query files; refuses to start if a required query is missing
src/validate.js	HttpError class and all input checks
src/env.js	Loads backend/.env from a fixed path
scripts/load-data.js	npm run load-data
scripts/export-snapshot.js	npm run export-snapshot: rewrites frontend/data.js from the database

8.2 Read endpoints

Method and path	Parameters	Query	Returns
GET /api/health	-	RETURN 1	{ok, database: up/down}
GET /api/meta/rooms	-	meta_rooms	blocks, floors, rooms
GET /api/meta/sections	-	meta_sections	sections with department, year, semester, cluster, schedule
GET /api/meta/faculty	-	meta_faculty	faculty list
GET /api/meta/periods	-	meta_periods	every period of every schedule
GET /api/search	q (1-40 characters)	search	sections, rooms, faculty, blocks
GET /api/floors/:floorId/occupancy	slot	floor_occupancy	each room: FREE or OCCUPIED with section, class, faculty
GET /api/floors/:floorId/grid	day, grid (optional)	floor_day_grid	rooms x periods of one schedule
GET /api/rooms/free	slot; building, floor, type, minCapacity optional	free_rooms	free rooms
GET /api/rooms/:roomId/week	-	room_week	classes of a room across the week
GET /api/sections/:id/week	-	section_week	section grid in its own schedule
GET /api/sections/:id/free	day optional	section_free_time	free periods per day
GET /api/sections/:id/free-with-rooms	day, floor	section_free_with_rooms	free periods with free classrooms of a floor
GET /api/sections/:id/courses	-	course_summary	courses with L-T-P, teacher, classes allotted vs needed
GET /api/faculty/:id/week	-	faculty_week	everything a teacher teaches (both schedules)
GET /api/faculty/:id/free	day, grid optional	faculty_free_time	free periods in the chosen schedule

8.3 Edit endpoints (Add data)

Method and path	Action
GET /api/edit/status, /api/edit/courses	Page asks whether editing is allowed; list of courses for the forms
POST /api/edit/rooms	Add or change a room (block, name, floor, number, type, seats); new block and floor are created
POST /api/edit/faculty	Add or change a faculty member (department, initials, name, cluster)
POST /api/edit/courses	Add or change a course; creates the L-T-P links and gives the course to every section of the semester
POST /api/edit/sections	Add or change a section; creates missing department/program/year/semester; sets its bell schedule; allots every course of the semester
POST /api/edit/teacher	Set (or clear) the course teacher of one section's course
POST /api/edit/classes	Add a class: checks periods, course part, rooms, faculty, then clashes; saved only if free
DELETE /api/edit/{classes|rooms|faculty|courses|sections}/:id	Remove; refused with a clear message while anything depends on the item. Removing the last room of a floor also removes the empty floor and block

How adding a class is checked (one transaction)

1. `class_lookup` finds the section, its schedule, the first period and the next back-to-back period, the allotment, and whether the course has that part (L, T or P).
2. Refused (400/404) if: no such section or period, not a working day, a two-period class cannot start before a break, the course is not in the section's semester, the course has no such part, a room or faculty member does not exist.
3. `class_clashes` lists every class already in those periods that uses the same section, a chosen room or a chosen teacher, *including overlapping periods of the other schedule*. If any exist the answer is **409** with each clash described, for example "MON P4: room 406 is used by 5E4 (DEV)".
4. Only if there is no clash, `class_create` creates the class and links it. Because everything is one transaction, a failed check saves nothing.

8.4 Input validation

Input	Rule
Day	MON, TUE, WED, THU, FRI or SAT
Slot	Optional schedule letter, day, -P and 1-9: MON-P4 or B-MON-P4
Schedule (grid)	One capital letter
Ids in the address	Letters, digits, dot, dash, underscore; 1-80 characters
Room type	CLASSROOM or LAB
Seats	Whole number 1-5000 (filter) or 1-1000 (form)
Search text	1-40 characters, passed as a parameter
Form text	Names 1-80 characters without control characters or < >
Codes	Block 'PJ'; room '409' / 'LAB 402'; department 2-8 letters; initials 1-6 letters; course code 3-15 letters/digits; L-T-P 'd-d-d'; year 'YYYY-YY' with a correct second part; section 1-12 letters/digits
JSON body	Maximum 20 KB; broken JSON gets a clear 400; non-JSON gets 415

Status codes: 400 bad input, 403 not allowed, 404 not found, 409 clash or still in use, 415 wrong content type, 413 too large, 503 database problem (with an explanation), 500 unexpected.

8.5 npm scripts

Command (in backend/)	Action
npm start	Start the server
npm run dev	Start with automatic restart on file changes
npm test	Run all tests (node --test, one file at a time)
npm run load-data	Load schema and both timetables
npm run export-snapshot	Rewrite frontend/data.js

9. The frontend (web page)

9.1 Design

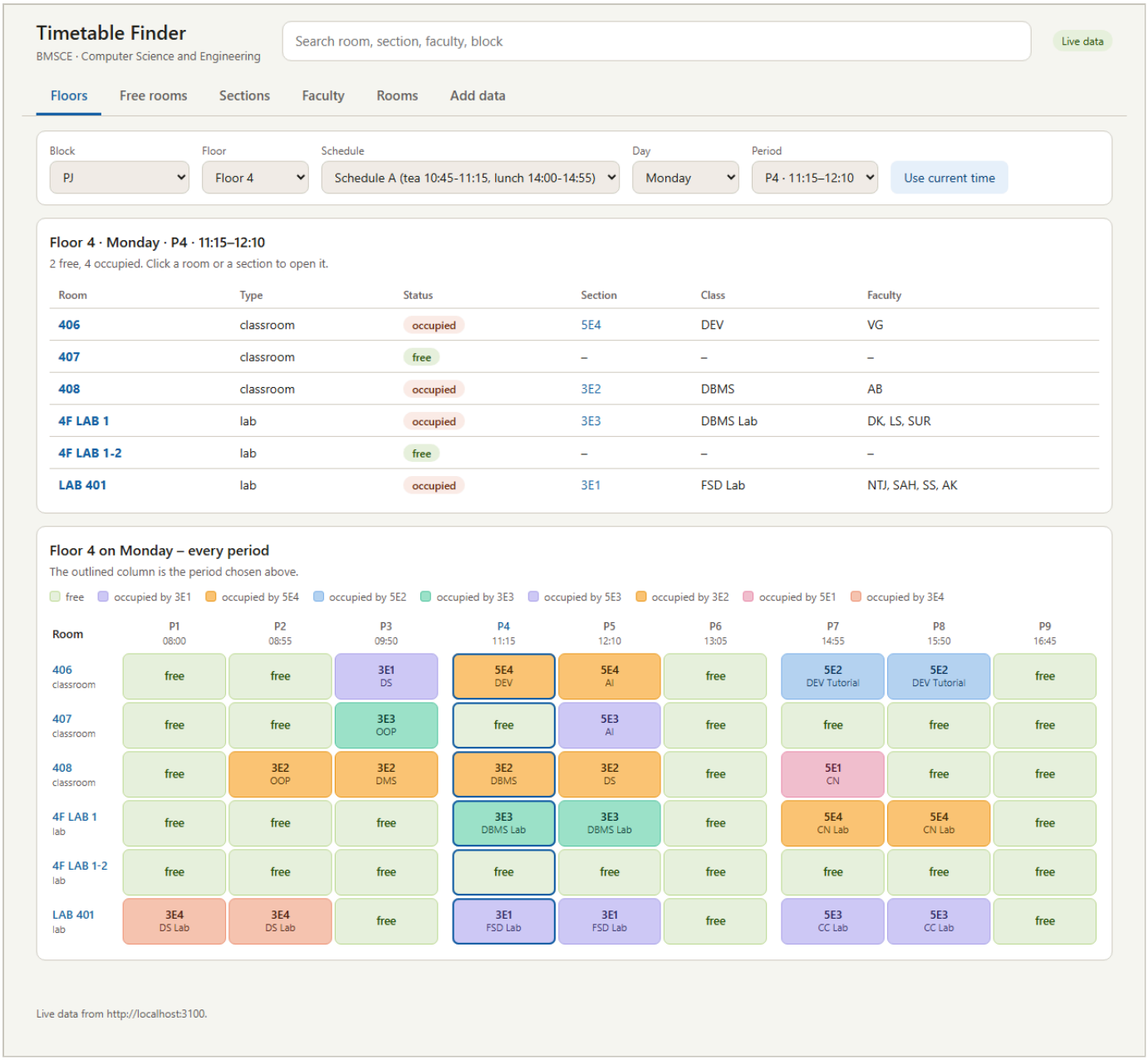
- **Plain HTML, CSS and JavaScript**, no framework and no build step. Opens in any modern browser.
- **Hash routing**: every filter is stored in the web address (for example `#/floors?b=PJ&f=PJ-F4&d=MON&p=4&g=A`), so a result can be shared as a link, and the back button works.
- **Two data layers behind one interface** (`window.TT_API` in `api.js`): *Remote* asks the backend; *Local* answers the same questions from the snapshot `data.js`. The tests prove both return the same answers. The badge at the top says *Live data* or *Snapshot data*.
- **Safe rendering**: elements are created with text content, never HTML strings.
- **Look**: light and dark mode follow the system; responsive down to phone width; coloured cells per section; two-period labs and tutorials are merged into one wide cell; break columns shown in the grids.
- **Accessibility**: labelled form fields, `aria-current` on the active tab, `aria-live` on the content area, table headers with scope.
- **Errors**: any problem is shown in a box with the real reason and a **Try again** button; an empty database says how to load the data.

9.2 Files

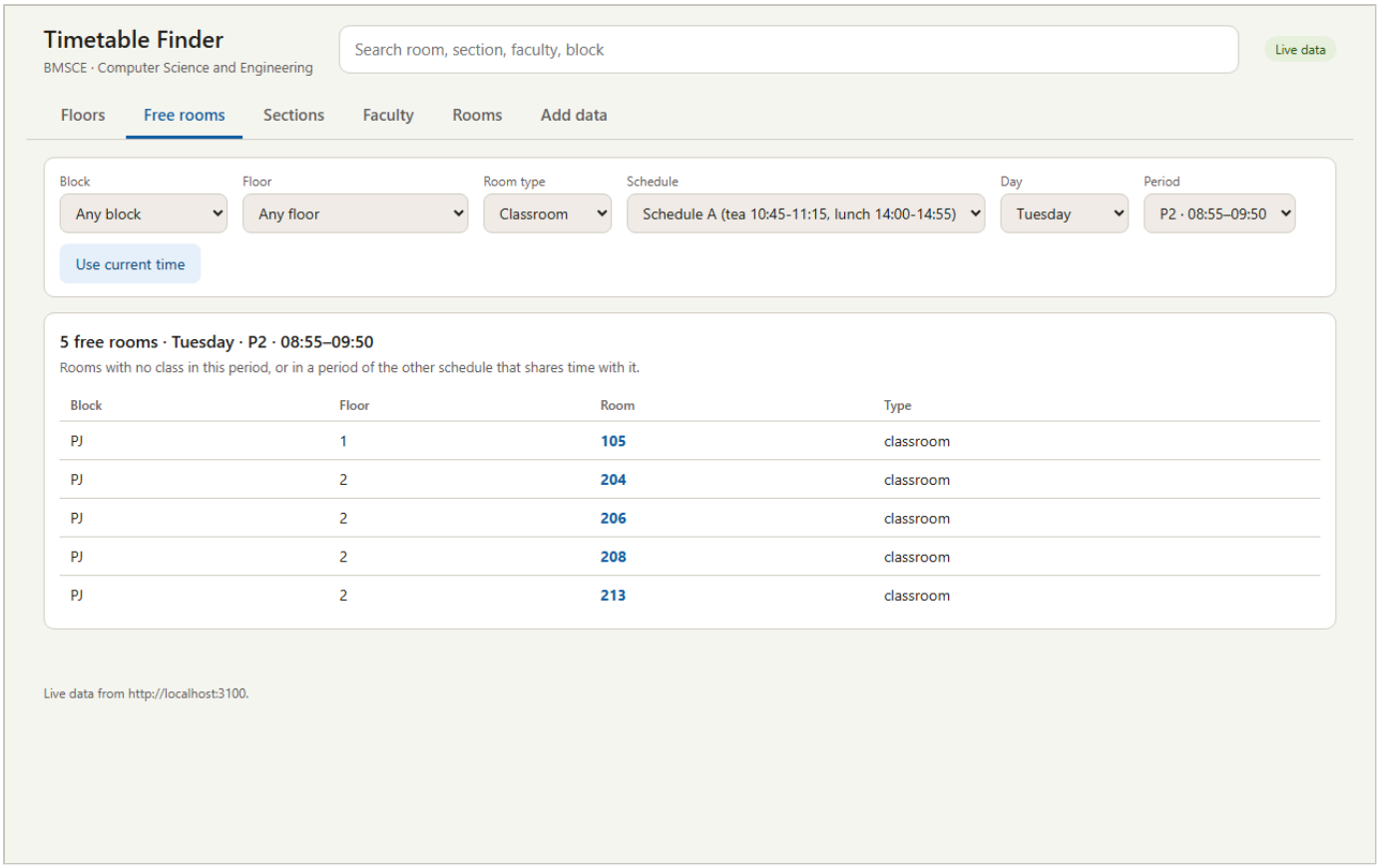
File	Role
index.html	Page skeleton, search box, tab bar, loads the four scripts
style.css	Colours as variables (light/dark), tables, grids, forms, phone layout
config.js	API_URL; the backend replaces it with a version that points at itself
api.js	Remote and Local data layers
app.js	Router, helpers, the five view screens, search, schedule handling
edit.js	The Add data tab (loaded before app.js, drawn by it)
data.js	Generated snapshot of the database (grids, slots, rooms, sections, faculty, courses, allotments, classes)

9.3 Screens

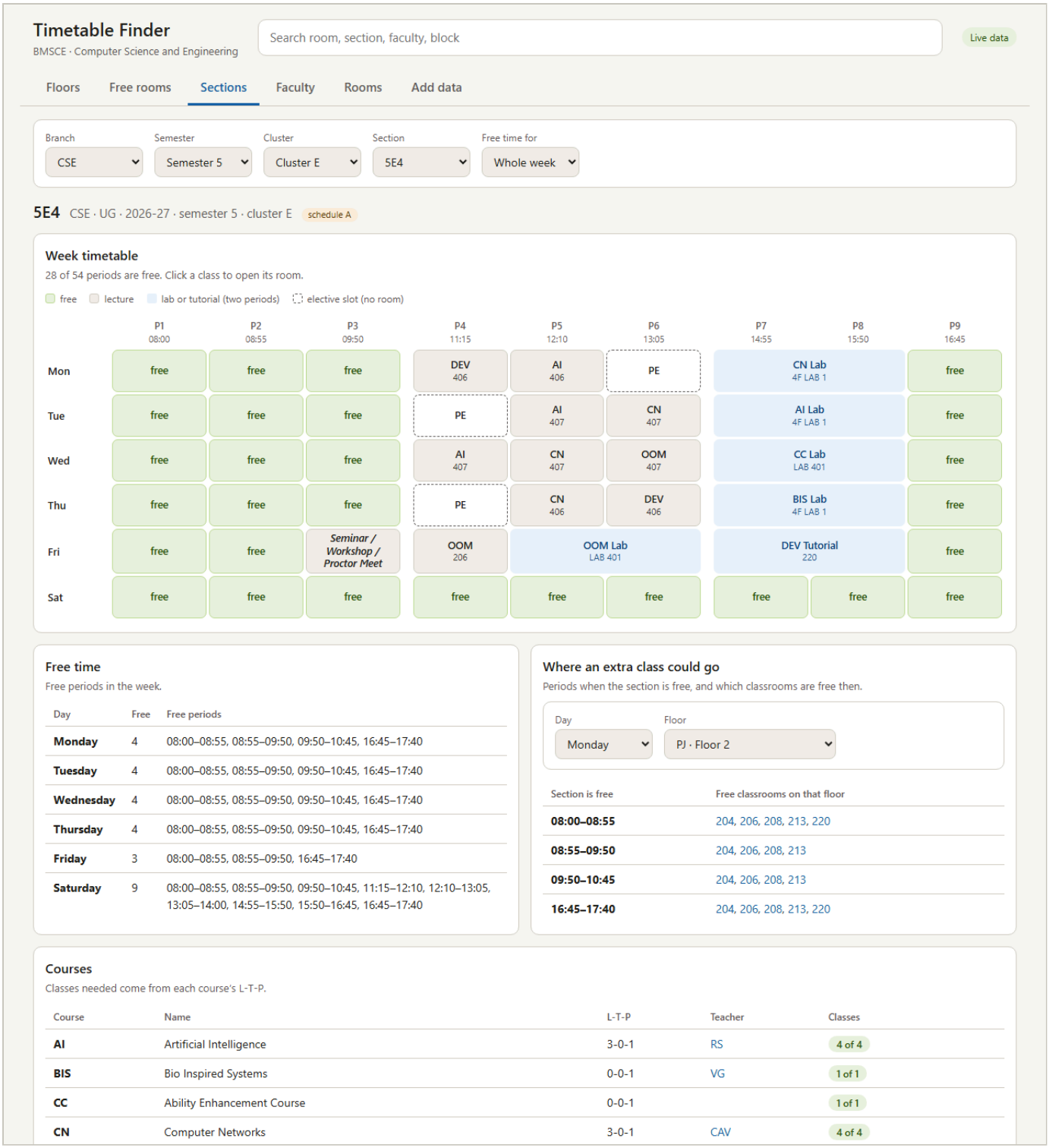
Tab	What you can do
Floors	Choose block, floor, schedule, day and period: every room is free or occupied by a named section; plus a rooms x periods grid for the whole day
Free rooms	Find free rooms by block, floor, room type, schedule, day and period; 'Use current time' button
Sections	Cascading pickers Branch, (Year), Semester, Cluster, Section; week grid with free periods in green; free-time summary; free classrooms for each free period of a chosen floor; course list with L-T-P and classes allotted vs needed
Faculty	Week timetable and free time of one teacher, in the schedule you pick; classes of the other schedule appear in the periods they share time with
Rooms	Pick block, floor, room (and schedule): which section uses the room and when
Add data	Forms (see 9.5)
Search box	Finds sections, rooms, faculty and blocks and opens them



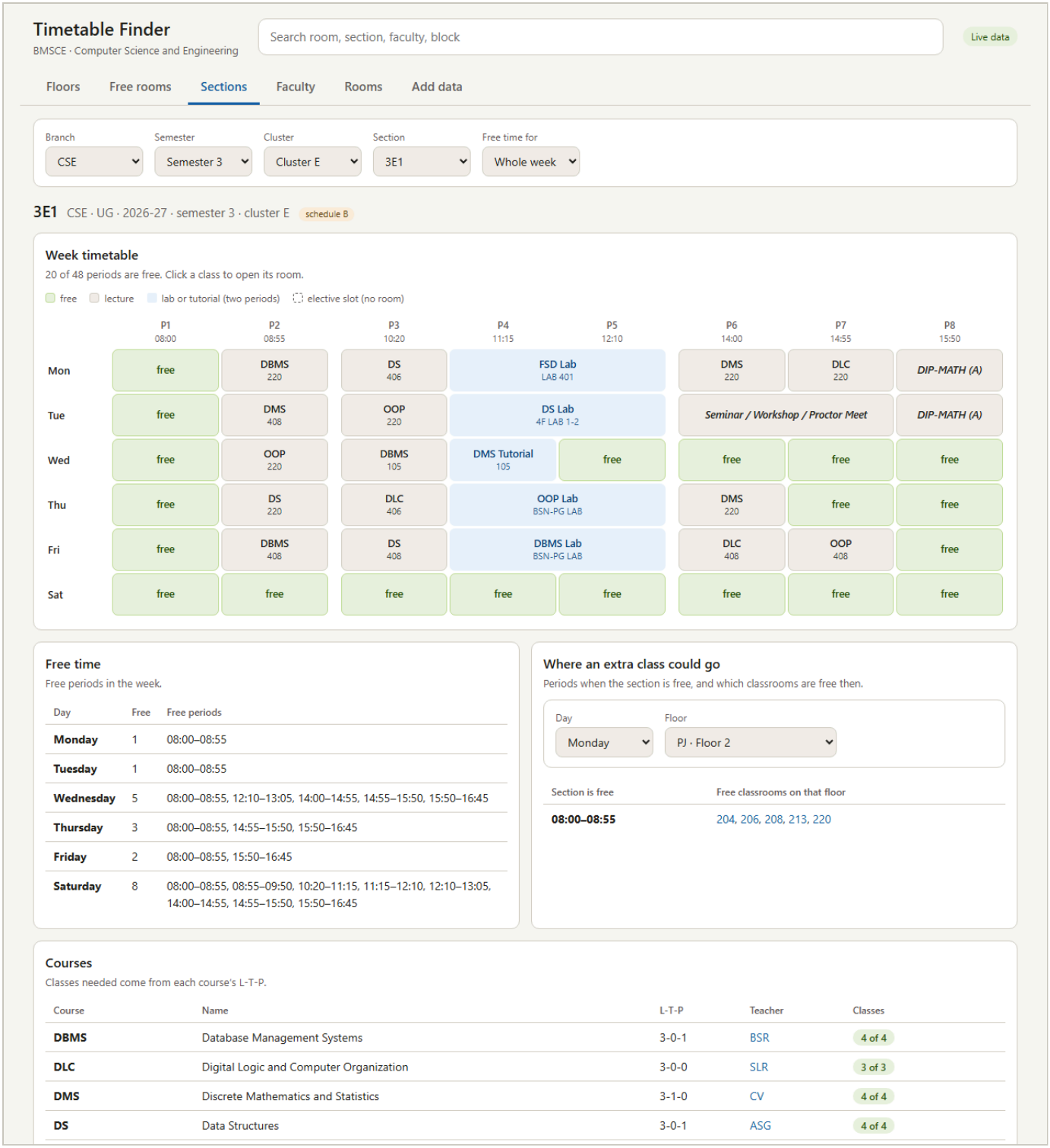
Floors tab: rooms of floor 4 in the PJ block, Monday period 4, with the whole-day grid



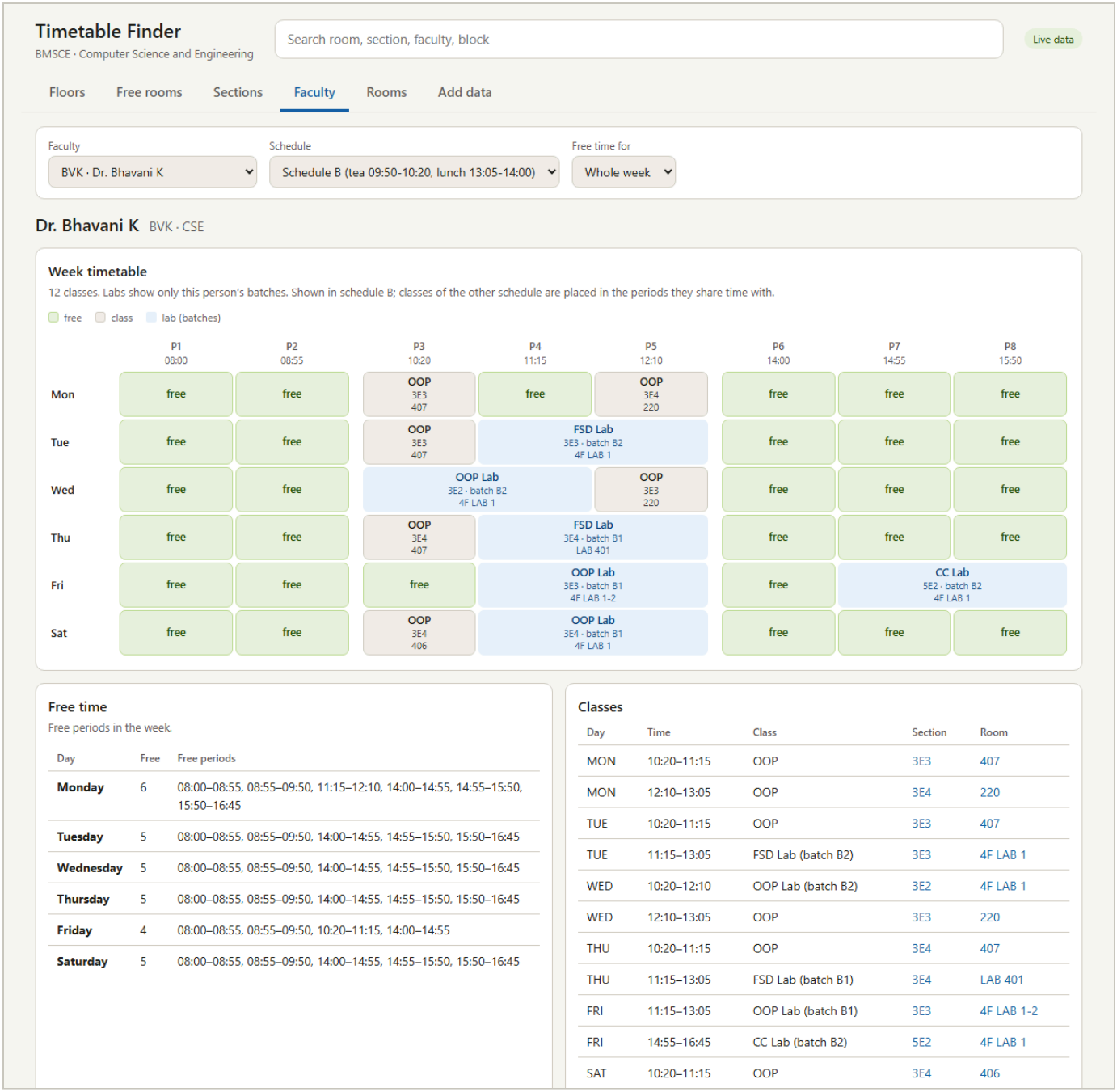
Free rooms tab: free classrooms on Tuesday in period 2 (schedule A)



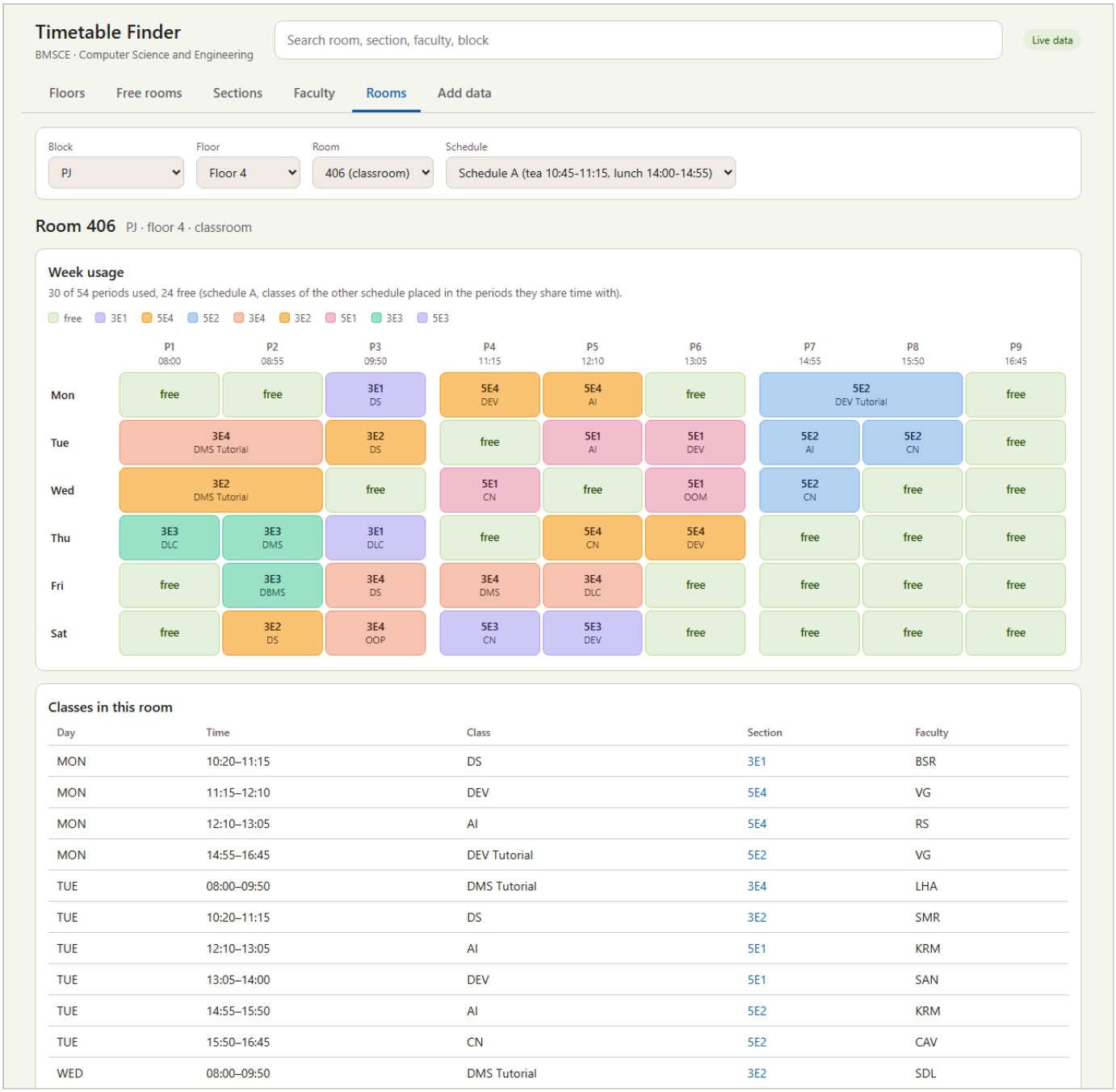
Sections tab: 5th semester section 5E4 (schedule A), free time, extra-class suggestions, course list



Sections tab: 3rd semester section 3E1 (schedule B): tea after P2, lunch after P5, DIP-MATH activities



Faculty tab: one teacher who teaches in both semesters, shown in schedule B



Rooms tab: room 406 across the week, with classes of both schedules placed in the periods they share time with

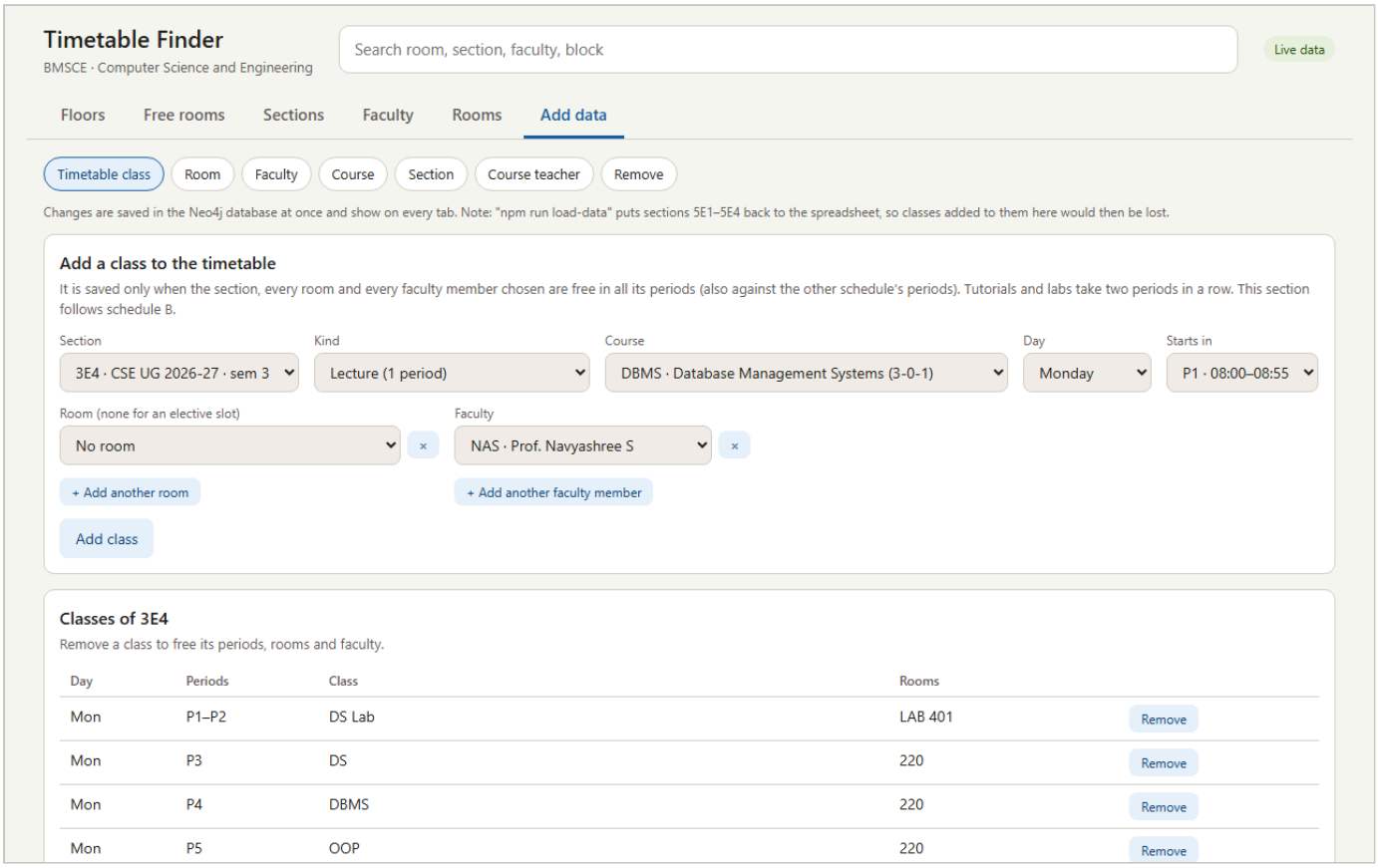
9.4 How schedules appear on screen

- A section's own screens always use its schedule (a small tag shows which).
- Floors, Free rooms, Rooms and Faculty have a **Schedule** drop-down because rooms and teachers are shared.
- A class from the other schedule is placed in every period of the displayed schedule that shares time with it (done in the page by comparing start and end times). A class that falls inside a break of the displayed schedule has no column; a note counts such classes and they remain in the list under the grid.

9.5 Add data tab

Form	Purpose
Timetable class	Add a lecture, tutorial, lab or activity: choose section, kind, course, day and starting period (periods of the section's own schedule), then rooms and faculty. Room and Faculty are drop-down lists; + Add another adds a further drop-down for labs with several rooms or teachers, and the cross removes one. The course teacher is pre-selected. Below the form is the section's class list with a two-click Remove button.

Form	Purpose
Room	Add or change a room; new block and floor are created
Faculty	Add or change a faculty member
Course	Add or change a course with its L-T-P; given to every section of the semester
Section	Add or change a section, including its bell schedule
Course teacher	Set who teaches a section's course
Remove	Delete a room, faculty member, course or section nothing uses any more



Add data tab: the class form with drop-down lists for room and faculty, and the section's class list

After every successful change the page reloads its lists and shows a confirmation with a link to the result. A refused change shows the server's reason under the form and keeps what was typed.

10. Testing and verification

10.1 Automated tests (98 in total, all passing at the last run on 23 Oct 2026)

File	Kind	What it checks
test/api.test.js	API and page	Health; read-only connection (the database refuses a write); every endpoint's answer equals the Local data layer on the same data (about 30 comparisons, including both schedules); lists; search finds only sections, rooms, faculty and blocks; schedules (9 vs 8 periods, section weeks); a room used in one schedule is busy in the overlapping period of the other; no clash anywhere; every 3rd-semester course has the classes its L-T-P needs; 14 kinds of bad input refused with 400; SQL-style injection text treated as plain text; page served with security headers and cannot be framed; README, .env and scripts are not served; friendly database messages
test/edit.test.js	Add data	Who may change data (this computer only, not another website, not another host name, JSON only); validation hints; add section, room, faculty, course; teacher; add a lecture and see it in the section, room and faculty views; clashes refused with every reason listed; labs take two periods and cannot start before a break; a section on schedule B uses its own periods and clashes with schedule A are found; unknown schedule refused and nothing saved; items in use cannot be removed; full removal. Uses throwaway ids starting with ZZT that are deleted afterwards
test/load-data.test.js	Loader	The .cypher splitter (comments, quotes, escapes) and that each data file splits into the expected number of statements, each starting with a Cypher keyword

Run with `npm test` in `backend/` with the database running. The comparison tests expect the loaded timetables; after manual edits through Add data, run `npm run load-data` and `npm run export-snapshot` first.

10.2 Independent verification against the spreadsheets

- **5th semester:** 80 of 80 sessions and 36 of 36 subject rows matched the spreadsheet.
- **3rd semester:** a separate script compared every session of the spreadsheet with what the API returns for each section: **116 of 116 period cells**, the room of each class and the faculty of each lab batch all match; nothing missing, nothing extra.
- **Clash checks:** no section, teacher or room has two classes at the same time, within a schedule or across the two (query 12 returns no rows).
- **Course needs:** every REGULAR course of every 3rd-semester section has exactly the number of classes its L-T-P requires.
- **Idempotent loading:** loading twice gives byte-identical snapshots.
- **Browser checks** in a real browser: every tab, search, error handling when the database is stopped, adding and removing a class and a room, drop-down lists, schedule switching, offline snapshot mode.

11. Running and operating the project

11.1 Start

Double-click **start_timetable_finder.bat** (or the **Timetable Finder** shortcut on the Desktop). It checks that Node.js and `backend/.env` exist, installs the backend packages the first time, starts the database in its own window (only if `.env` points at port 17687), waits up to 3 minutes for it, then starts the server, which opens **http://localhost:3100**. The first start takes about a minute. Keep both windows open while using the page; close them to stop.

Requirements: Node.js 20 or newer, and Java 21 (it comes with Neo4j Desktop; the launcher finds it automatically).

11.2 Using Neo4j Browser

Open http://localhost:17474, choose Username / Password and use the values from `backend/.env`. Useful commands: `CALL db.schema.visualization()` to draw the schema; any query from `04_queries.cypher`.

11.3 Troubleshooting

You see	Cause and fix
Cannot reach the Neo4j database	The database is not running. Close the windows and start the launcher again
The database refused the login	NEO4J_USER / NEO4J_PASSWORD in backend/.env do not match the database
The database holds no timetable yet	Run npm run load-data in the backend folder
Cannot reach the Timetable Finder server	The server window was closed; start the launcher again
Port 3100 is already used	Set PORT=3101 in backend/.env and open http://localhost:3101
Database window says Java is missing/wrong version	Install Neo4j Desktop (brings Java 21) or Java 21 and set JAVA_HOME
Badge says Snapshot data	index.html was opened directly; open http://localhost:3100 for live data

11.4 Typical tasks

Task	How
Add a class, room, teacher, course or section	Add data tab (changes are saved at once)
Reset to the spreadsheets	npm run load-data
Update the offline copy after data changes	npm run export-snapshot
Add the next timetable	Write a generator like build_cluster_E_sem3_cypher.py (or use the Add data forms), then load it
Use Neo4j Desktop or Aura	Change the NEO4J_* lines in backend/.env, run npm run load-data once
Show the page to other devices	Add HOST=0.0.0.0 to backend/.env (viewing only; editing stays on the host computer)

12. Problems found and fixed

These came from code review, automated tests and real browser testing.

Area	Problem	Fix
Database	Driver waited about 30 s when the database was stopped	Short timeouts; clear message in about 7 s
Database	Login refused: the Neo4j Desktop password did not match	Project's own database with a generated password; Desktop/Aura switch documented
Database	Whole numbers stored as decimals (semester 1.0)	Whole numbers sent as Neo4j integers (found by a test)
Database	OVERLAPS links missing for a period with itself (a pattern never matches the same period twice), so classes in the same period were invisible to busy checks	Self-links added in the schema; found by the comparison tests
Backend	.env was read from the current folder	Fixed path in env.js
Backend	Port 3000 clashed with another application	Default port 3100 everywhere
Backend	A section saved with an unknown schedule would still be created	Check moved inside the transaction
Backend	Slot names in messages looked like 'B MON-P1'	All dashes replaced
Frontend	Colours not applied (CSS specificity)	Selectors made specific
Frontend	Search results for courses opened an arbitrary section	Courses removed from search
Frontend	'Use current time' could pick a non-working day	Day checked first
Frontend	Block shown as 'OTHER' instead of its name	Block name returned and used
Frontend	Empty or stopped database showed 'Failed to fetch' or crashed	Clear messages and a Try again button
Frontend	A stray 'null' appeared in form messages	Messages built without null children
Launcher	Processes started from tools died when the session turn ended; 'timeout' fails without a console; batch files need Windows line endings	Start through Explorer; ping used to wait; CRLF files
Security	No headers, no origin check on editing	CSP and friends; edit guard (local only, Host and Origin checks)
Project	Obsolete seed files, old PDFs, two launchers, Python tools that only worked without a login	Removed (to the Recycle Bin); files renumbered; documentation rewritten
Diagram	A class can use several rooms (LAB 503 + LAB 504)	Class diagram corrected; schedule classes added

13. Assumptions and limitations

13.1 Assumptions that should be checked against the original timetables

- Blocks and floors of rooms: all numbered rooms and "LAB nnn" are put in block PJ on the floor given by the first digit. Only 406 and 407 were confirmed as PJ.
- "ISE-SH" and "BSN-PG LAB" are in a block called "Location not given" (floor 0) because no location is given.
- "4F LAB 1-2" is kept as *one* room. If it means labs 1 and 2 together, it overlaps the use of "4F LAB 1" by other sections and clash checks cannot see that.
- Classes written "LAB 503,504" use two rooms at once.
- The four mathematics teachers of the 3rd semester have no initials in the sheet; initials were made from their names and they were placed in a Mathematics department.
- DIP-MATH (A) and (B) are stored as one-period activities; Skill Lab-1 and NCMC are "not timetabled" because the sheet has no session for them.
- The 5th semester has no EVS lecture in the sheet; every 5th semester section shows EVS as 0 of 1 classes until one is added (Add data or the optional auto-allotment).
- Everyone with initials belongs to the CSE department and cluster E.

13.2 Limitations

- Only cluster E of the CSE department is loaded. The model scales to the whole college, but other clusters need their data converted.
- Students are not loaded; the procedure for 10,000+ students is in `03_load_students.cypher`.
- The page has no user accounts. Anyone who can reach the server can view; only the host computer can edit.
- A room used by two overlapping classes shows only the first in the floor screens (this is a clash and is reported by query 12).
- Room capacity (seats) is optional and mostly empty, so seat-based filtering is not shown.
- The offline snapshot is a copy: it must be regenerated after data changes.
- The launcher is for Windows. The backend and page run anywhere Node.js runs.
- `neo4j/01_schema_graph.pdf` was drawn before schedules were added and does not show TimeGrid; the class diagram is current.

13.3 Possible next steps

- Load the remaining semesters and clusters; add student data.
- User login (for example for HOD or timetable-coordinator roles) before putting the page on a network.
- Timetable import from the Add data tab (spreadsheet upload), export of a section or teacher timetable to PDF.
- Seat capacities for rooms and a "find a room for N students" screen.

14. Repository and delivery status

Item	Status
Repository	https://github.com/makkenavineela-dev/Timetable-Finder (main branch had only a one-line README)
Local commit	f969cd6 on top of the existing 'Initial commit': 42 files (about 720 KB)
Excluded from Git	backend/.env and backend/.env.* (database password), local-neo4j/ (the 170 MB database), node_modules/, logs, local Claude settings. backend/.env.example (no password) is included
Push	Not yet done: no GitHub login was available to the assistant. Run <code>git push -u origin main</code> in the project folder and sign in as the owner of the repository; Git Credential Manager opens a sign-in window

After cloning the repository on another computer: install Node.js and Java 21, run `npm install` in `backend/`, create `backend/.env` from `.env.example` and point it at a Neo4j database (or unpack a Neo4j 5.26 Community copy as `local-neo4j/` and set its password), run `npm run load-data`, then start the application.

Appendix A. Optional auto-allotment script (07_auto_allot.cypher)

Places the classes a section still needs, from each course's L-T-P. It is **not** part of `npm run load-data` (it would re-add classes removed by hand). Run it in Neo4j Browser when needed; it only creates what is missing, so it is safe to repeat.

Rules: labs first, then tutorials, then lectures (hardest to fit first). A class goes into the first block of periods where the section is free in every period; every teacher (batch teachers for labs) is free; a room of the right type is free (home room, then same floor, then any); the course has no other class of the same type that day; the day is a working day and the periods are back-to-back in the **section's own schedule**, with room and faculty also free in overlapping periods of the other schedule. Among the possible places, the lightest day of the section and the earliest period win.

```
// ======================================================================
// Automatic class allotment from L-T-P. OPTIONAL: the timetable files (02_*) already hold the
// printed timetables; run this by hand to place classes that are still missing, for example
// the EVS lecture or a new section. It is NOT part of npm run load-data, because it would
// re-add classes you removed on the page. Safe to re-run: it only creates what is missing.
// Run it in Neo4j Browser (http://localhost:17474).
// ======================================================================
//
// The number of weekly classes for each course comes from its HAS_LTP
// relationships to the L, T and P nodes (Part B):
//   L credits x 1 period  -> one-period lectures
//   T credits x 2 periods -> two-period tutorials
//   P credits x 2 periods -> two-period labs (all batches together, one lab room)
// Only missing classes are created, so it is safe to re-run. After loading the 02_* files the
// gaps are the EVS lecture (1-0-0) of each 5th semester section and courses without a teacher.
//
// A class goes into the first block of periods where:
//   - the section is free in every period of the block
//   - every faculty member taking it is free (batch faculty for labs)
//   - a room of the right type is free (home room, then same floor, then any)
//   - the course has no other class of the same type that day
//   - the day is a working day and the periods are back-to-back, in the section's own schedule
//     (TimeGrid); room and faculty must also be free in every OVERLAPPING period of other schedules
// Among those, the lightest day of the section and the earliest period win.
// Labs go first, then tutorials, then lectures (hardest to fit first).
// A REGULAR course with no teacher yet is skipped. Anything that cannot be
// placed stays pending in query 1 of 04_queries.cypher.
// ======================================================================

// Step 1 - make sure every section has an Allotment for every course of its semester
MATCH (sem:Semester)-[:HAS_SECTION]->(sec:Section), (sem)-[:HAS_COURSE]->(c:Course)
MERGE (a:Allotment {id: sec.id + '-' + c.code})
MERGE (sec)-[:HAS_ALLOTMENT]->(a)
MERGE (a)-[:OF_COURSE]->(c);


// Step 2 - allot the missing classes of every section in the college.
// To allot only one section, change the first line to:
//   MATCH (sec:Section {id:'CSE-UG-2026-27-S5-5E4'})-[:HAS_ALLOTMENT]->(a:Allotment)-[:OF_COURSE]->(c:Course)
MATCH (sec:Section)-[:HAS_ALLOTMENT]->(a:Allotment)-[:OF_COURSE]->(c:Course)
WHERE c.scheduling IN ['REGULAR', 'ELECTIVE_SLOT']
MATCH (c)-[req:HAS_LTP]->(ct:LTP)
OPTIONAL MATCH (a)-[:HAS_CLASS]->(done:Class)-[:OF_TYPE]->(ct)
WITH sec, a, c, ct, req.classesPerWeek AS needed, count(done) AS allotted
WHERE allotted < needed
UNWIND range(1, needed - allotted) AS n
WITH sec, a, c, ct, n
ORDER BY ct.priority, a.id, n
CALL (sec, a, c, ct) {
  // 1. who must be free: the faculty of every batch for a lab, the course teacher otherwise
  OPTIONAL MATCH (a)-[:HAS_BATCH]->(:Batch {type:'LAB'})-[:ASSIGNED_TO]->(bf:Faculty)
  OPTIONAL MATCH (tf:Faculty)-[tr:TEACHES]->(a)
  WHERE tr.role = 'COURSE_TEACHER'
  WITH sec, a, c, ct, collect(DISTINCT bf) AS labStaff, collect(DISTINCT tf) AS teachers
  WITH sec, a, c, ct, CASE WHEN ct.byBatch THEN labStaff ELSE teachers END AS staff
  WHERE c.scheduling = 'ELECTIVE_SLOT' OR size(staff) > 0
```

```
  // 2. candidate blocks: periodsPerClass back-to-back periods of the section's schedule on a working day
  MATCH (sec)-[:FOLLOWS]->(:TimeGrid)-[:HAS_SLOT]->(t1:TimeSlot)<-[:HAS_SLOT]-(d:Day {working: true})
  MATCH p = (t1)-[:NEXT*0..2]->(:TimeSlot)
  WHERE length(p) = ct.periodsPerClass - 1
  WITH sec, a, c, ct, staff, d, t1, nodes(p) AS block

  // 3. keep blocks where the section, the faculty and a suitable room are all free
  WHERE NOT EXISTS { MATCH (sec)<-[:FOR_SECTION]-(:Class)-[:AT_TIME]->(t) WHERE t IN block }
    AND NOT EXISTS { MATCH (f:Faculty)<-[:TAUGHT_BY]-(:Class)-[:AT_TIME]->(:TimeSlot)-[:OVERLAPS]-(t)
                     WHERE f IN staff AND t IN block }
    AND NOT EXISTS { MATCH (a)-[:HAS_CLASS]->(prev:Class)-[:OF_TYPE]->(ct),
                           (prev)-[:AT_TIME]->(:TimeSlot)<-[:HAS_SLOT]-(d) }
    AND (c.scheduling = 'ELECTIVE_SLOT'
         OR EXISTS { MATCH (r:Room {type: ct.roomType})
                     WHERE NOT EXISTS { MATCH (r)<-[:IN_ROOM]-(:Class)-[:AT_TIME]->(:TimeSlot)-[:OVERLAPS]-(t) WHERE t
IN block }
                       AND (r.capacity IS NULL OR sec.strength IS NULL OR r.capacity >= sec.strength) })

  // 4. lightest day of the section first, then the earliest period
  WITH sec, a, c, ct, staff, d, t1, block,
       size([(sec)<-[:FOR_SECTION]-(:Class)-[:AT_TIME]->(s:TimeSlot)<-[:HAS_SLOT]-(d) | s]) AS dayLoad
  ORDER BY dayLoad, d.order, t1.period
  LIMIT 1

  // 5. room: home room first, then a room on the same floor as the home room, then any free room
  OPTIONAL MATCH (sec)-[:HOME_ROOM]->(home:Room)<-[:HAS_ROOM]-(homeFloor:Floor)
  OPTIONAL MATCH (fl:Floor)-[:HAS_ROOM]->(r:Room {type: ct.roomType})
  WHERE c.scheduling = 'REGULAR'
    AND NOT EXISTS { MATCH (r)<-[:IN_ROOM]-(:Class)-[:AT_TIME]->(:TimeSlot)-[:OVERLAPS]-(t) WHERE t IN block }
    AND (r.capacity IS NULL OR sec.strength IS NULL OR r.capacity >= sec.strength)
  WITH sec, a, c, ct, staff, t1, block, r,
       CASE WHEN r = home THEN 0 WHEN fl = homeFloor THEN 1 ELSE 2 END AS roomRank
  ORDER BY roomRank, r.id
  LIMIT 1

  // 6. create the class and link it to its periods, faculty, room and batches
  MERGE (cl:Class {id: a.id + '-' + ct.code + '-' + t1.id})
    SET cl.name = CASE ct.code WHEN 'L' THEN c.short
                               WHEN 'T' THEN c.short + ' Tutorial'
                               ELSE c.short + ' Lab' END
  MERGE (a)-[:HAS_CLASS]->(cl)
  MERGE (cl)-[:OF_TYPE]->(ct)
  MERGE (cl)-[:FOR_SECTION]->(sec)
  FOREACH (t IN block | MERGE (cl)-[:AT_TIME]->(t))
  FOREACH (f IN staff | MERGE (cl)-[:TAUGHT_BY]->(f))
  FOREACH (rm IN CASE WHEN r IS NULL THEN [] ELSE [r] END | MERGE (cl)-[:IN_ROOM]->(rm))
  WITH a, ct, cl, block, r
  OPTIONAL MATCH (a)-[:HAS_BATCH]->(b:Batch {type:'LAB'})
  WHERE ct.byBatch
  WITH cl, block, r, collect(b) AS batches
  FOREACH (b IN batches | MERGE (cl)-[:FOR_BATCH]->(b))
  RETURN cl.name AS class, [t IN block | t.id] AS slots, r.id AS room
}
RETURN sec.name AS section, class, slots, room;
```